

Utilizando UML para projetar o software

Na fase de projeto de sistemas de informação, usar diagramas de linguagem unificada de modelagem (UML) é importante para entender e planejar o software. Esses diagramas incluem interação, classes de projeto, atividades, estado, componentes e implantação, ajudando a visualizar e estruturar o sistema de forma clara e organizada.

Prof. Sylvio Jorge

Preparação

Antes de iniciar o conteúdo, é indicado que seja instalado em seu computador um programa que lhe permita elaborar modelos, sob a forma de diagramas da UML (linguagem unificada de modelagem). Nossa sugestão inicial é o Astah Free Student License e será necessário usar seu e-mail institucional para ativar a licença. Preencha os dados do formulário no site do software, envie e aguarde a liberação de sua licença em seu e-mail. Ao recebê-la, siga as instruções e instale o produto em seu computador.Sugestões de links adicionais de programas livres para modelagem de sistemas em UML (UML Tools) podem ser encontradas em buscas na internet.

Objetivos

- Empregar os diagramas de interação no projeto de sistema.
- Revisar o diagrama de classes da análise no projeto de sistema.
- Empregar os diagramas de estado e de atividades no projeto de sistema.
- Empregar os diagramas de componentes e de implantação.

Introdução

A modelagem de um software pode, de fato, determinar a quantidade de problemas que terão que ser tratados durante o desenvolvimento. Softwares de sucesso, que têm sua implantação quase sem nenhum tipo de erro, certamente contaram com um bom processo de modelagem durante seu desenvolvimento. Ou seja, um bom produto de software é sempre concebido por um bom processo de desenvolvimento, iniciado com a modelagem do sistema.

O processo de desenvolvimento conta com diversos métodos e técnicas para que o produto tenha a melhor qualidade possível, sendo a modelagem uma das técnicas utilizadas. Modelar algo é construir uma representação abstrata de um artefato do mundo real.

A modelagem conta com diversas regras e diretrizes que têm que ser respeitadas para se construir um modelo de qualidade para o software. Todas as funcionalidades implementadas pelo software, assim como todos os elementos de dados que utilize e as regras que devem ser seguidas, precisam ser pensadas, discutidas e estruturadas antes do início de sua construção, de modo que haja um completo entendimento de todo esse universo, não só por parte dos usuários, mas também dos desenvolvedores. Saber identificar cada característica do software durante seu projeto é importante para que venha de fato atender às necessidades dos usuários e cumprir com todas as regras a serem executadas por ele.

Vamos entender como construir os diagramas ao longo da etapa de projeto (ou design) de software. Nesta parte do desenvolvimento, alguns outros diagramas já terão sido construídos e precisam ser respeitadas suas regras e definições. Para isso, vamos ver as técnicas de concepção e de modelagem desses diagramas, assim como seus atributos e modelos. Por fim, vamos apresentar e discutir como aplicá-los.

O vídeo Introdução, a seguir, vai apresentar como utilizar UML para projetar o software, com destaque para o emprego dos diagramas de interação, classes, estado, atividades, componentes e de implantação na etapa de projeto.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Diagramas de comunicação e de sequência na UML

Em sistemas computacionais orientados a objetos, as funcionalidades são realizadas por meio da colaboração. Os objetos interagem, executando, assim, os casos de uso. Na UML, essas interações são representadas por diagramas de interação, que mostram o comportamento dinâmico do sistema ao descrever o fluxo de mensagens entre objetos.

Existem dois tipos principais: **diagramas de interação de comunicação e de sequência**. Eles ajudam a detalhar e refinar outros modelos, fornecendo aos programadores uma visão clara dos objetos e mensagens envolvidos nos casos de uso do sistema.

Neste vídeo, conheça as atividades típicas de um processo de desenvolvimento de software e os cenários possíveis de aplicação dos diagramas de interação.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Atividades do desenvolvimento de software

Independentemente do processo adotado, as atividades típicas de um processo de desenvolvimento de software, geralmente, compreendem as seguintes etapas:

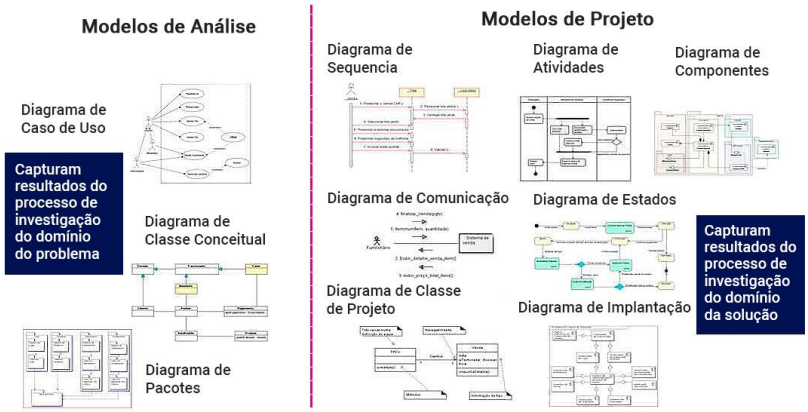
1. Levantamento de requisitos
2. Análise
3. Projeto (design)
4. Implementação
5. Testes
6. Implantação

Essas atividades são realizadas por etapas ou fases, em que uma etapa depende dos artefatos resultantes das anteriores. Neste conteúdo, focaremos na etapa de projeto (ou design) do sistema, na qual existe uma dependência de seus artefatos em relação aos artefatos elaborados na fase de análise (ou modelagem).

A imagem a seguir ilustra os diagramas da UML que, usualmente, são construídos nos modelos de análise e de projeto. Trata-se de uma visão prática, uma vez que a especificação da linguagem UML não determina quais diagramas devam ser usados em quais etapas e, tampouco, em que ordem devem ser elaborados. Observe!

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Modelos da fase de análise X Modelos da fase de projeto.

Diagramas de interação

Como você já sabe, em sistemas computacionais orientados a objetos, os casos de uso são realizados a partir da colaboração entre grupos de objetos que se comunicam.

Comentário

Na UML, as comunicações são representadas por diagramas de interação, que mostram o comportamento interativo de um sistema. **Os diagramas de interação descrevem o fluxo de mensagens e fornecem contexto para uma ou mais linhas da vida de objetos dentro de um sistema**. Além disso, os diagramas de interação podem ser usados para:

- Representar as sequências organizadas dentro de um sistema.
- Servir como um meio para visualizar dados em tempo real.

Os diagramas de interação podem ser implementados em diversos cenários para fornecer um conjunto exclusivo de informações. Conheça algumas possibilidades de uso:

- Modelar um sistema, como uma sequência de eventos organizada por tempo.
- Fazer a engenharia reversa ou avançada de um sistema ou processo.
- Organizar a estrutura de vários eventos interativos.
- Mostrar, de forma simples, o comportamento de mensagens e linhas da vida dentro de um sistema.
- Identificar possíveis conexões entre elementos de linhas da vida.

Comentário

Engenharia reversa, no contexto da Engenharia de Software, significa obter o modelo a partir do sistema implementado.

Podemos, então, dizer que seus principais objetivos são fornecer:

Informações adicionais

Obter informações adicionais para completar e aprimorar outros modelos (modelo de casos de uso e de classes).

Visão detalhada

Fornecer aos programadores uma visão detalhada dos objetos e das mensagens envolvidos na realização dos casos de uso do sistema.

Além disso, há dois tipos de diagrama de interação. Vamos conhecê-los!

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

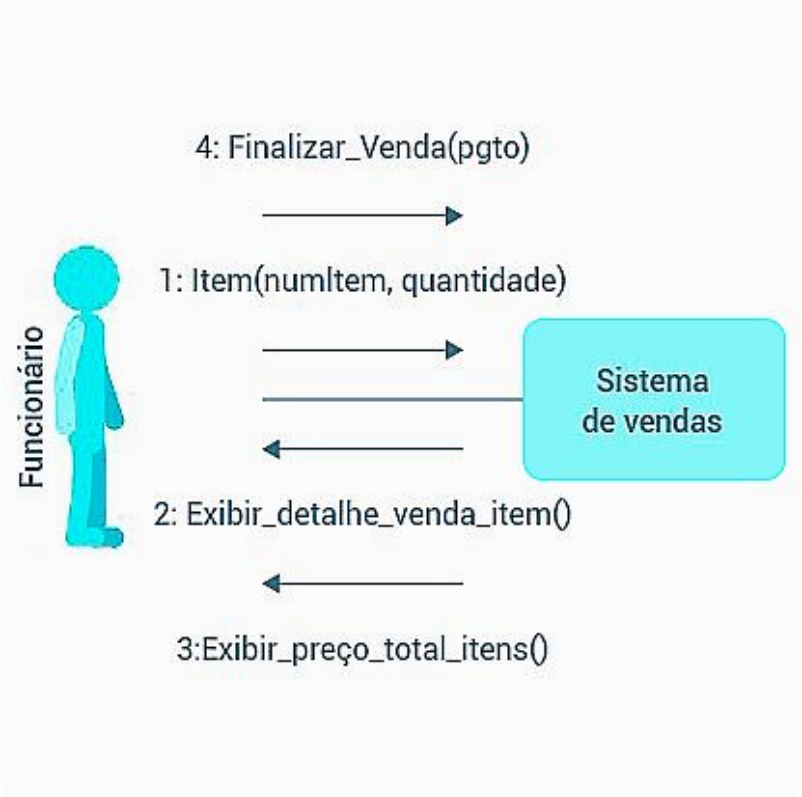


Diagrama de comunicação.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

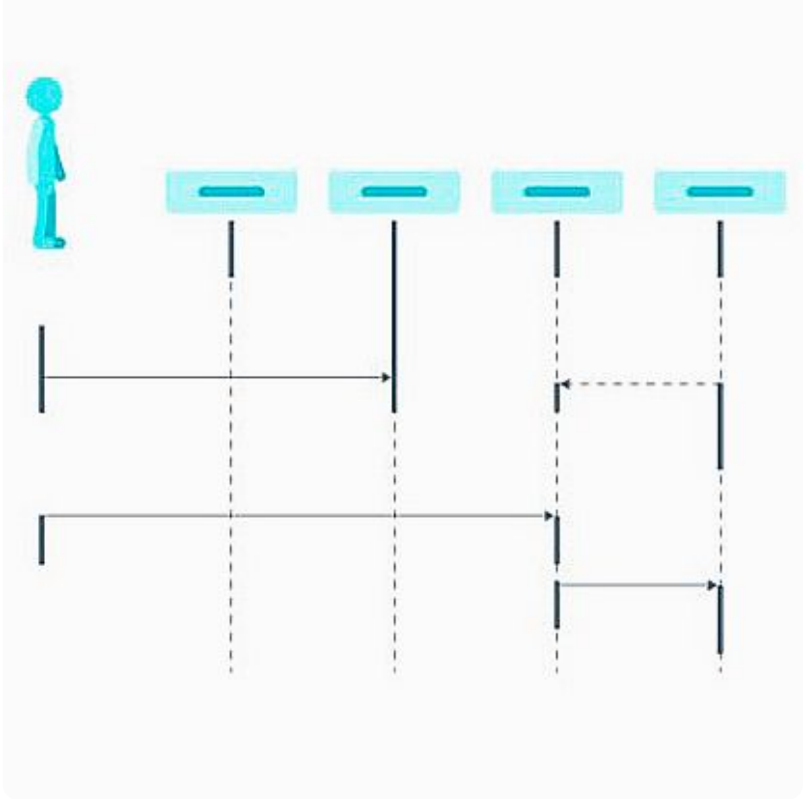


Diagrama de sequência.

Cada tipo de diagrama aborda uma perspectiva diferente do comportamento de um sistema. Em um diagrama de interação, as mensagens trocadas entre objetos, em geral, serão métodos da classe a que pertence o objeto receptor da mensagem, com os mesmos elementos oriundos das mensagens. Isso será percebido ao estudarmos os diagramas de classe de projeto.

> Os diagramas de interação servem para refinar o diagrama de classes na etapa de projeto, com o detalhamento dos métodos.

Os diagramas da etapa de projeto apresentados na imagem anterior estão relacionados entre si e com os diagramas da etapa de análise, de modo que a construção de um diagrama depende de outros. Vamos ver, a seguir, os conceitos básicos de cada diagrama, como construí-los e utilizá-los.

Atividade 1

Os grupos de objetos interagem através de comunicações em um sistema computacional orientado a objetos (SOO). Qual é o principal objetivo dos diagramas de interação na UML em sistemas orientados a objetos?

A Identificar a quantidade de objetos necessários no sistema.

B Representar o comportamento dinâmico e o fluxo de mensagens entre objetos.

C Descrever apenas a estrutura estática dos objetos.

D Avaliar o desempenho do sistema em tempo real.

E Listar todos os atributos e métodos de cada objeto.

A alternativa B está correta.

Os diagramas de interação na UML são usados para mostrar como os objetos do sistema se comunicam ao longo do tempo para realizar casos de uso específicos. Eles detalham o fluxo de mensagens entre os objetos, o que é essencial para compreender o comportamento dinâmico do sistema. Esses diagramas ajudam a visualizar e organizar a sequência de eventos interativos, fornecendo contexto para as linhas de vida dos objetos e auxiliando no refinamento dos modelos de classes e casos de uso.

Diagramas de sequência e de comunicação na UML

São ferramentas essenciais para descrever como os objetos de um sistema de informação interagem. Enquanto os diagramas de sequência mostram a ordem temporal das mensagens trocadas entre objetos, os diagramas de comunicação destacam as associações e as mensagens trocadas, focando as conexões entre os objetos. Ambos são utilizados para documentar casos de uso complexos e validar operações das classes.

Neste vídeo, conheça o emprego dos diagramas de sequência e de comunicação ao longo do processo de desenvolvimento de software, bem como seus principias elementos.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Diagrama de sequência

Diagramas de sequência recebem esse nome porque descrevem as comunicações entre objetos de um sistema de informação, ao longo de uma linha do tempo. Em geral, são construídos logo após os diagramas de caso de uso e de classes, pois têm como principais objetivos:

- Documentar casos de uso.
- Mostrar como os objetos do sistema se comunicam através de mensagens em ordem temporal.
- Validar se todas as operações das classes foram identificadas e declaradas.
- Validar a existência de um objeto necessário ao funcionamento do sistema.

Atenção

Não é necessário construir diagramas de sequência para todos os casos de uso do sistema. Apenas os mais complexos, relacionados com o negócio da aplicação, são explorados por esse tipo de diagrama. Ele pode ser bastante trabalhoso e criá-lo para todas as partes do sistema pode não ser produtivo e nem ter benefícios compensatórios.

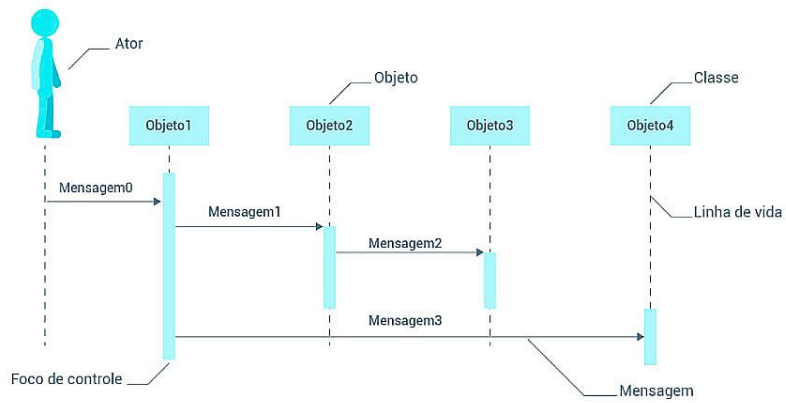
Dependendo da complexidade ou do número de objetos envolvidos ou da presença de desvios condicionais no caso de uso, vários diagramas de sequência serão necessários para representar as interações do sistema. Assim, devemos construir um diagrama de sequência principal, descrevendo as sequências normais entre os objetos (fluxo normal) e diagramas complementares, descrevendo sequências alternativas (fluxos alternativos) e tratamento de situações de erro (fluxos de exceção).

> Diagramas de sequência são complementares aos diagramas de caso de uso.

Os diagramas de caso de uso são bastante completos nas descrições textuais, porém não são capazes de mostrar como os objetos do sistema se comunicam por intermédio de mensagens em ordem temporal, nem verificar se todas as operações das classes foram validadas e declaradas. Essas funções são muito mais bem executadas pelos diagramas de sequência.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo esquemático de diagrama de sequência.

Elementos do diagrama de sequência

O diagrama de sequência é composto de diversos tipos de elementos, que você conhecerá, a seguir!

Objetos

São apresentados na parte superior do diagrama (um ao lado do outro) na dimensão horizontal (vide Exemplo esquemático de diagrama de sequência). Não há uma ordem específica de apresentação obrigatória. Sua ordem é relevante somente como forma de tornar o diagrama mais legível.

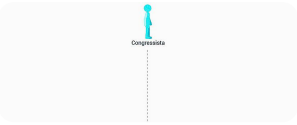
Dica

Uma boa prática pode ser colocar os elementos que têm mais interação mais próximos. Já no centro e abaixo dos objetos temos sua linha da vida (linha reta tracejada).

Todo e qualquer objeto ou classe nesse diagrama pode ser representado com um retângulo com o seu nome, mas existem notações específicas para tipos especiais de objetos. São quatro tipos especiais, a saber:

Ator

É uma entidade externa que interage com o sistema enviando ou recebendo dados. Um ator é representado pelo estereótipo de boneco magro (“palito”), como nos diagramas de casos de uso. Mensagens podem ser trocadas entre atores, ou entre atores e objetos ou classes de fronteira.



Fronteira

É uma classe ou um objeto responsável pela interface entre o sistema de informação e seus usuários, ou pela comunicação entre dois ou mais sistemas. Em geral, são programas (formulários, interfaces, janelas) que capturam, processam dados e exibem informações. É através da classe de fronteira que os atores (usuários) terão seus requisitos atendidos pelo sistema.



Controle

É uma classe ou um objeto usado quando os desenvolvedores optam por separar as regras de negócio, as manipulações de dados e o sequenciamento de tarefas da interface de comunicação do sistema com o usuário, seguindo um padrão de projeto conhecido como MVC (do inglês *Model – View – Controller*). Dessa forma, a classe de fronteira (*View*) só existe como interface para receber e exibir dados. Todas as outras tarefas são executadas pela classe de controle (*Controller*).



Entidades

São classes ou objetos responsáveis por armazenar e gerenciar os dados do sistema, bem como por mostrar a estrutura lógica e estática dos dados (Model no padrão MVC).

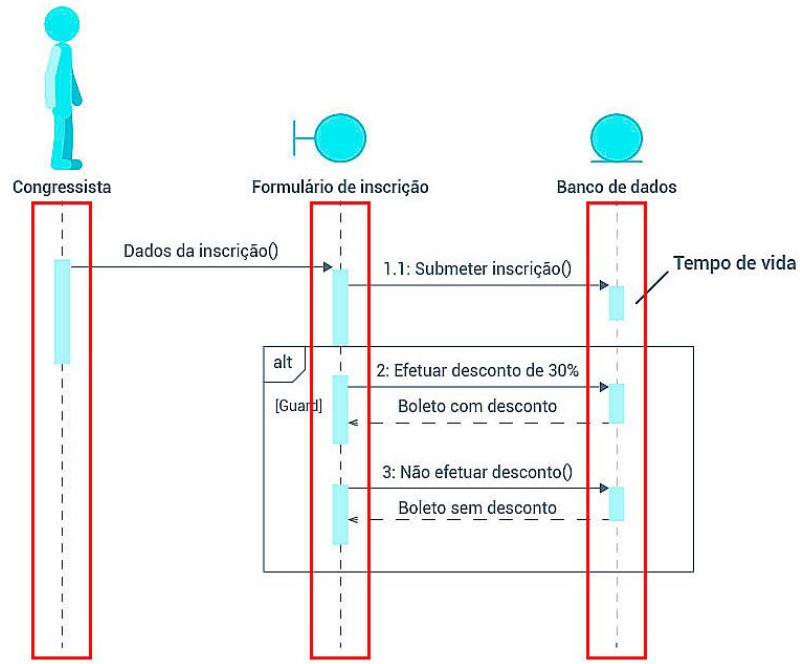


Linha de vida

Compõe a dimensão vertical (tempo) de cada objeto do diagrama. As linhas de vida representam a sequência na qual a vida do objeto transcorre durante a interação. Podem apresentar criação ou destruição de objetos. Quando são criadas interações entre os objetos, um retângulo sobre a linha é colocado (automaticamente pelas ferramentas de desenho de diagrama), identificando a interação. Mais de um retângulo pode existir em um único objeto. Esse retângulo é chamado de **tempo de vida da interação**.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplos de linhas de vida (destacados em vermelho).

Mensagens

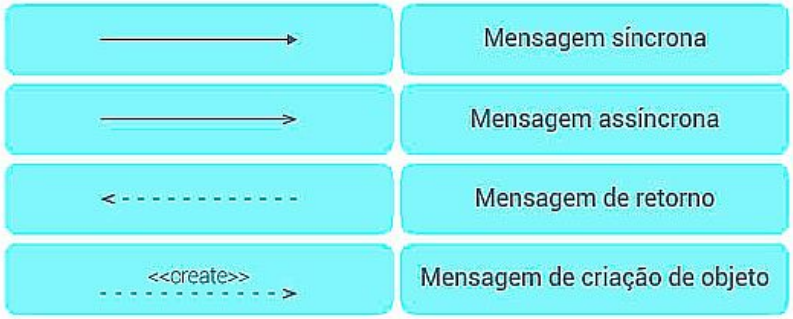
São os serviços solicitados de um objeto a outro ou a ele mesmo, e as respostas a essas solicitações. Uma mensagem é representada por um segmento de reta, contendo uma seta em uma das suas extremidades. As mensagens são acompanhadas de um rótulo contendo seu nome. Podem conter também parâmetros da mensagem. Elas podem ser dos seguintes tipos:

- Mensagens síncronas.
- Mensagens assíncronas.
- Mensagens de retorno.
- Mensagens de criação de objeto.

Observe que tipo de mensagem cada tipo de seta representa:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Tipos de setas X Tipos de mensagens.

Para cada uma delas, a linguagem usa notações diferentes. Na UML, o rótulo de uma mensagem deve seguir a seguinte sintaxe:

```
[[expressão-sequência] controle:] [v :=] nome [(argumentos)]
```

Os colchetes [ ] indicam que o elemento é **opcional**. Note que o único termo obrigatório corresponde ao nome da mensagem.

Muitas vezes é necessário indicar que o termo controle está associado a uma expressão lógica. Nesse caso, a sintaxe para o termo controle pode ser uma condição ou uma iteração:

```
‘*’ ‘[’ cláusulaiteração ‘]’

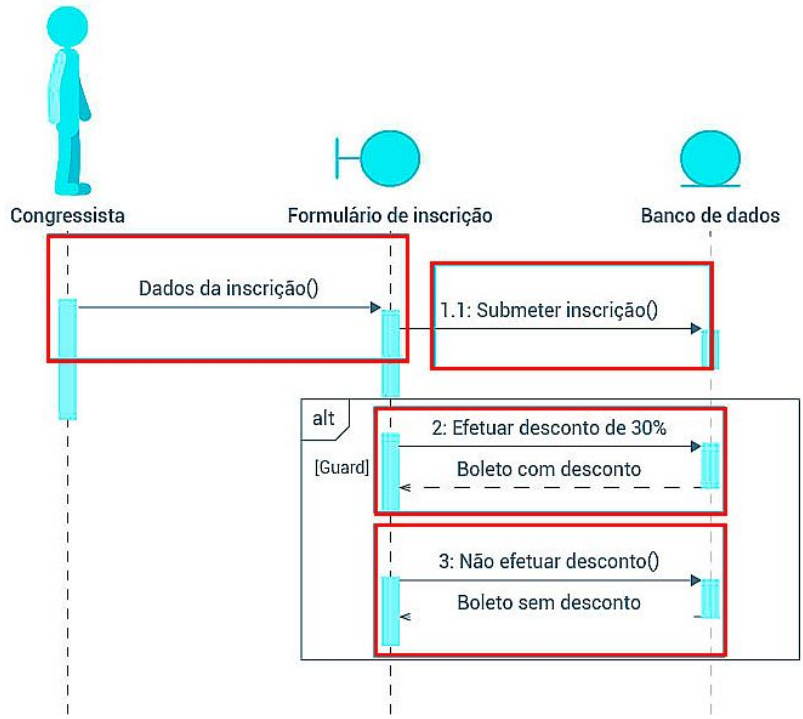
‘[’ cláusulacondição ‘]’
```

O símbolo * significa repetições da cláusula iterativa. Nessa sintaxe, os elementos entre aspas simples ‘ ‘ significam que fazem parte da expressão.

Observe alguns exemplos de mensagens destacadas em vermelho:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

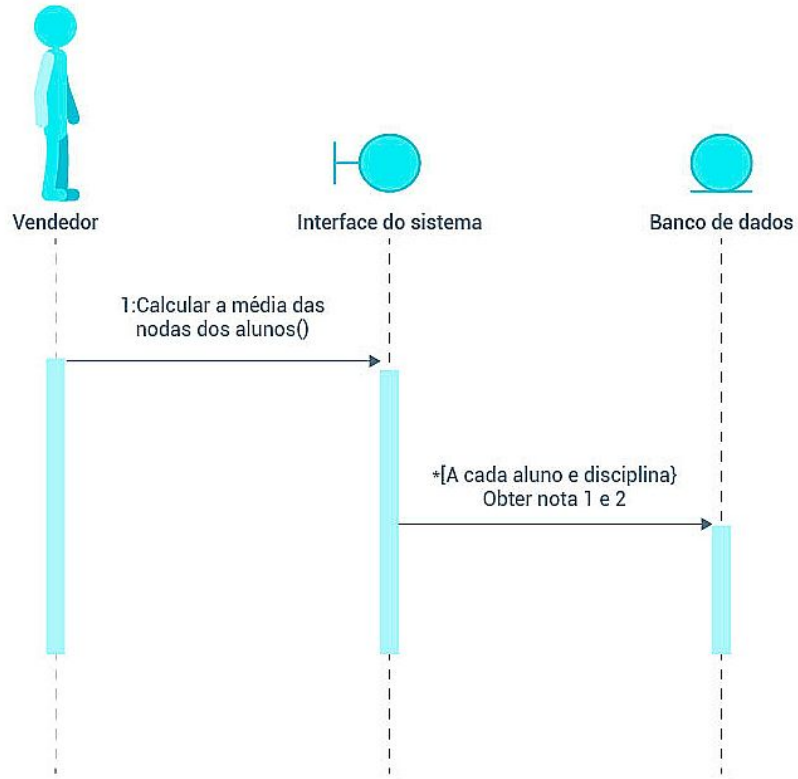


Exemplos de mensagens.

Mensagens de iteração

São mensagens que podem ser enviadas repetidas vezes. São precedidas de um asterisco e representadas dentro de colchetes.

> **Conteúdo interativo**
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.

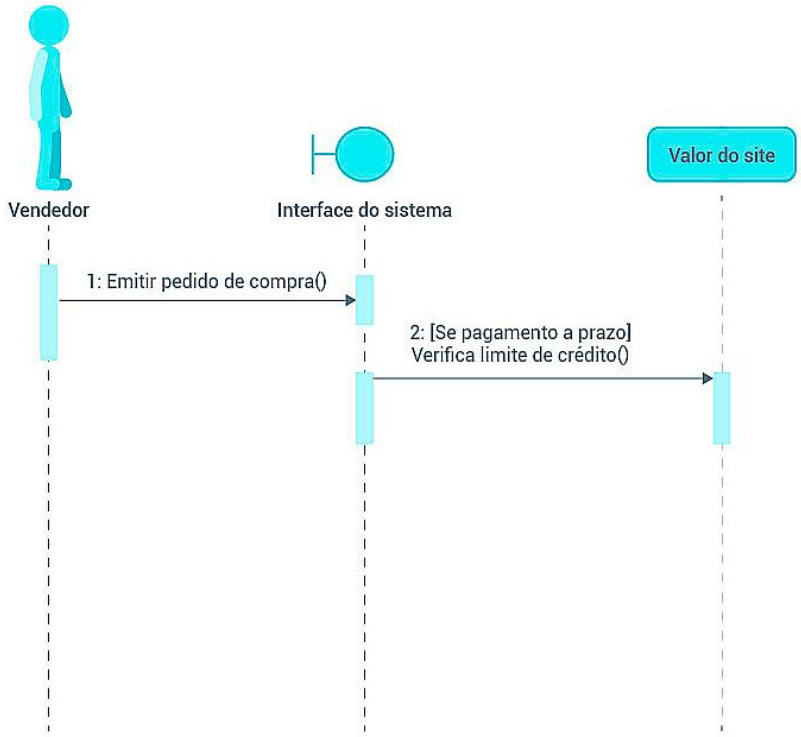


Exemplo de mensagens de iteração.

Condições de guarda

São mensagens que têm uma condição indicada para seu envio.

> **Conteúdo interativo**
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.

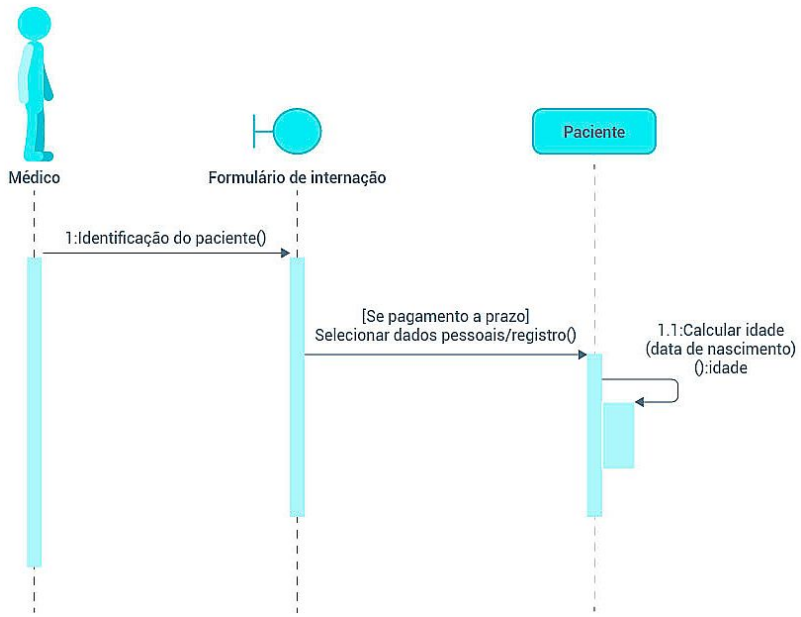


Exemplo de condições de guarda.

Autochamada

São mensagens que partem do objeto de origem e atingem a linha de vida dele mesmo disparando operações.

> **Conteúdo interativo**
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.

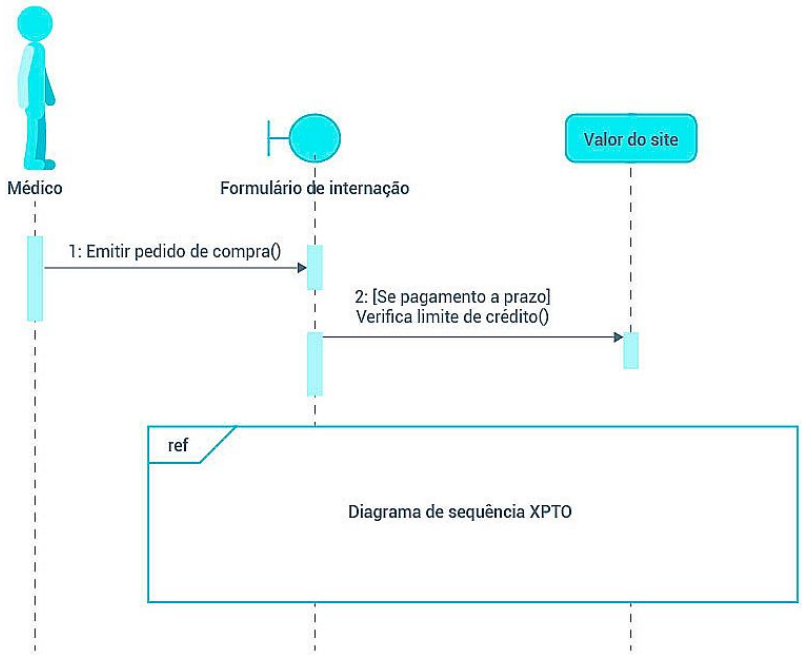


Exemplo de autochamada.

Ocorrência de interação

São elementos de modelagem úteis quando se deseja simplificar um diagrama e extrair uma porção para outro, ou quando existe uma porção de ocorrência que se possa fazer reuso em outro diagrama. É representado por uma moldura com a etiqueta “ref” que se refere a outro diagrama de sequência.

> Conteúdo interativo
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.

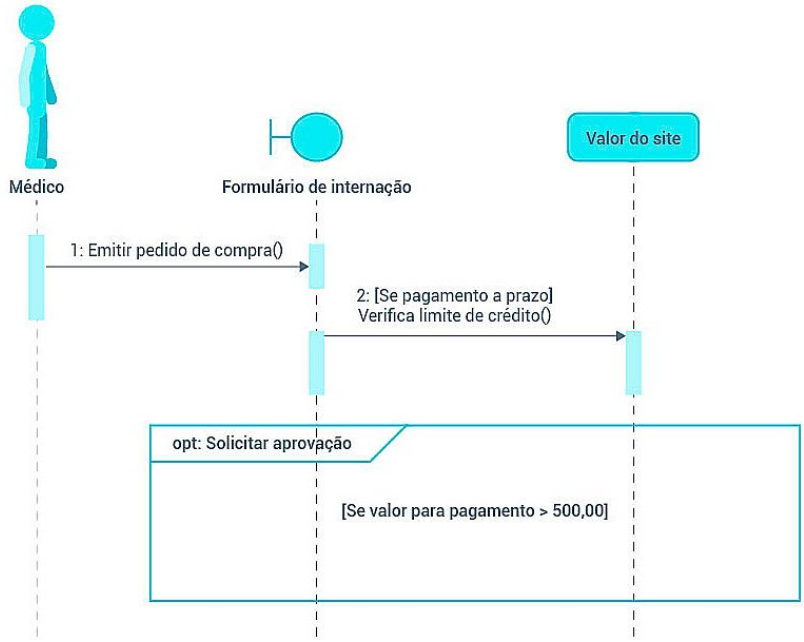


Exemplo de ocorrência de interação.

Opt

São elementos que permitem tornar um trecho da interação opcional.

> Conteúdo interativo
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.



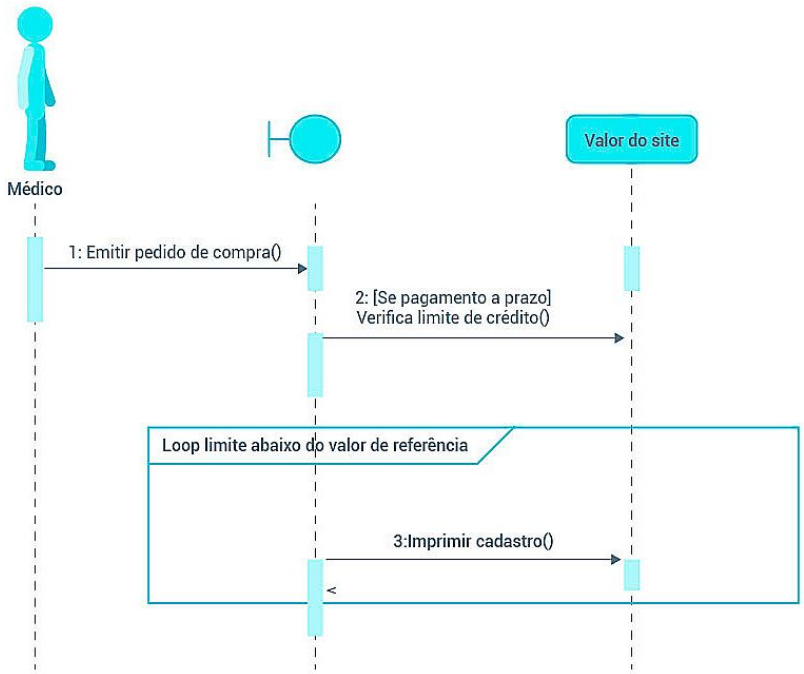
Exemplo de interação opcional “opt”.

Loop

São elementos que permitem repetir um trecho da interação até “n” vezes.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



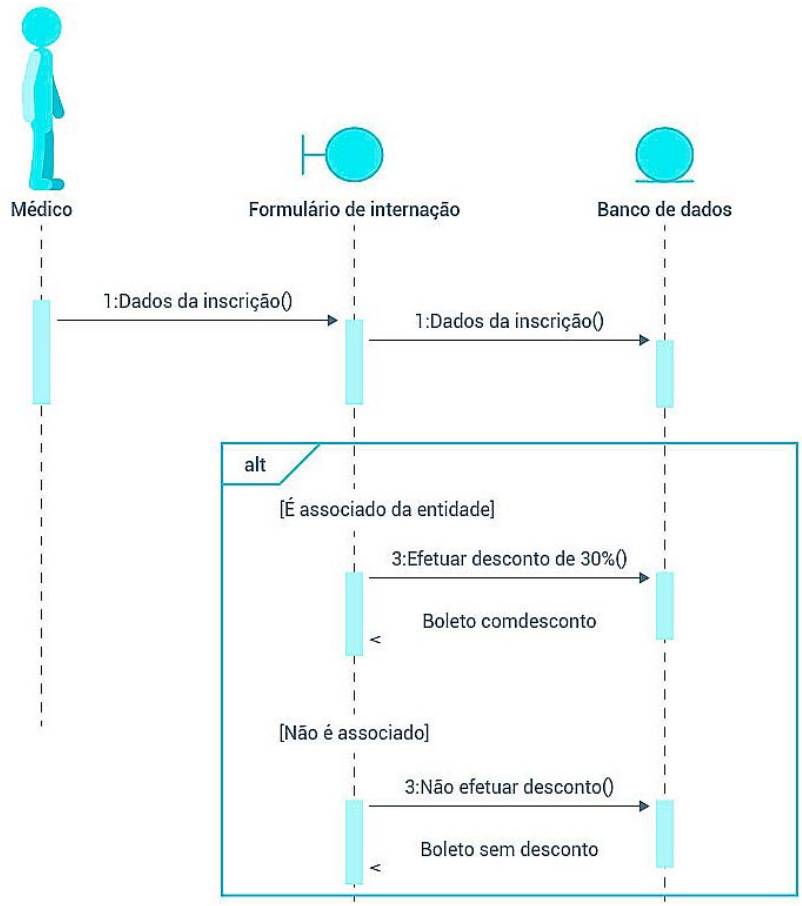
Exemplo de interação repetida “loop”.

Alt

São elementos que permitem que um trecho da interação seja alternativo.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de interação alternativa “Alt”.

Diagrama de comunicação

Diagramas de comunicação servem para mostrar como os objetos interagem para executar o comportamento de um caso de uso ou de parte dele, a fim de

- Determinar interfaces.
- Determinar responsabilidades de classes.
- Definir os papéis dos objetos que executam determinado fluxo de eventos.

Portanto, os diagramas de comunicação representam o fluxo de mensagens que ocorre entre os objetos de um sistema de informação, assim como o diagrama de sequência. Os dois expressam informações semelhantes, mas numa forma de exibição diferente. Vamos ver quando aplicar cada um?

Diagrama de sequência

Se o objetivo for apresentar o fluxo de mensagens no decorrer do tempo, utilize o diagrama de sequência.

Diagrama de comunicação

Se a intenção for de dar ênfase a como os objetos estão vinculados e quais mensagens trocam entre si, use o diagrama de comunicação.

No diagrama de comunicação, um vínculo é uma associação que identifica a ligação entre dois objetos envolvidos em um processo e é caracterizado pelo envio ou recebimento de uma mensagem ou de ambos.

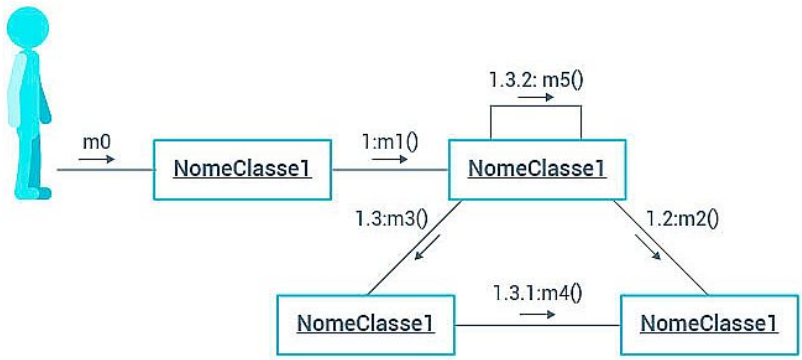
Comentário

As mensagens enviadas de um objeto para outro são representadas por segmentos de retas com uma seta em uma das extremidades, indicando sua direção. Estas mensagens devem ter nome, parâmetros (opcional) e a sequência que é expressa por números, assim como no diagrama de sequência.

Observe como um diagrama de comunicação deve se apresentar no exemplo a seguir!

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo esquemático de diagrama de comunicação correspondente ao diagrama de sequência, anteriormente apresentado.

O exemplo esquemático apresentado ilustra a correspondência que existe entre o diagrama de sequência e o diagrama de comunicação. Os mesmos objetos são envolvidos, trocando as mesmas mensagens.

Atenção

O fluxo de execução das mensagens, que no diagrama de sequência é determinado pela ordem temporal de cima para baixo, aqui no diagrama de comunicação é determinado pela numeração ordenada das mensagens.

Atividade 2

Há dois tipos de diagrama de interação. Qual é a diferença principal entre os diagramas de sequência e os diagramas de comunicação na UML?

A Diagramas de sequência são usados para definir a hierarquia de classes, enquanto diagramas de comunicação não.

B Diagramas de sequência descrevem a ordem temporal das comunicações, enquanto diagramas de comunicação enfatizam as conexões entre objetos.

C Diagramas de comunicação são usados apenas para modelar sistemas pequenos, enquanto diagramas de sequência são usados para sistemas grandes.

D Diagramas de sequência representam a estrutura estática dos objetos, enquanto diagramas de comunicação representam o comportamento dinâmico.

E Diagramas de sequência são utilizados apenas na fase de implementação, enquanto diagramas de comunicação são usados na fase de projeto.

A alternativa B está correta.

Os diagramas de sequência destacam a sequência temporal das mensagens trocadas entre objetos, detalhando o fluxo de interações ao longo do tempo. Em contraste, os diagramas de comunicação se concentram nas associações entre os objetos e nas mensagens trocadas, fornecendo uma visão clara de como os objetos estão vinculados e como interagem entre si.

Construção de um diagrama de interação

Para construir um diagrama de interação, seja de sequência ou de comunicação, é necessário seguir passos específicos: determinar o caso de uso, identificar os objetos envolvidos e suas conexões, definir a sequência de mensagens e seus tipos, e usar um software UML apropriado para criar o diagrama. Esses diagramas ajudam a visualizar a dinâmica entre objetos e a refinar modelos de casos de uso e de classes, tornando a interação do sistema mais clara e compreensível.

Este vídeo apresenta a construção de um diagrama de interação, bem como em que momento essa construção ocorre ao longo do processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Como construir um diagrama de interação?

Para criação de um diagrama de interação (sequência ou comunicação), você pode seguir os seguintes passos (em geral):

1. Determine o caso de uso que será modelado e identifique suas operações.
2. Para cada operação, identifique os objetos que farão parte de sua interação.
3. Identifique as conexões e os relacionamentos entre eles e, em seguida, categorize-os.
4. Identifique a sequência de fluxos de mensagens na interação entre os objetos.
5. Identifique os tipos de mensagens, bem como sua sequência.
6. Em seu software de diagramas UML, selecione as formas apropriadas para criar o diagrama.
7. Insira um rótulo para todas as formas para representar cada evento, interação e mensagem em seu sistema.
8. Nomeie o diagrama com algo que o identifique claramente, para facilitar na sua busca.

As classes conceituais identificadas, que participam em cada caso de uso, correspondem às entidades do mundo real envolvidas na tarefa do caso do uso, como se este fosse executado manualmente. Durante a modelagem de interações, objetos dessas classes participam da realização de um ou mais casos de uso. Por outro lado, durante a modelagem de interações, o projetista pode ter a necessidade de adicionar classes de software (ou seja, classes da aplicação que não têm correspondência no mundo real) que ajudem a organizar as tarefas a serem executadas. Essas classes de software normalmente são necessárias para manter a coesão e o acoplamento das demais classes em um nível adequado.

Dica

Não se recomenda incluir o ator nem o objeto de fronteira primeiramente, porque isso dificulta o entendimento. É mais recomendado iniciar a modelagem da interação com a representação do recebimento da mensagem para ativação da operação de sistema, no caso de uso. Essa recomendação ajuda a separar a lógica do domínio e a lógica da interface com o usuário.

Durante a aplicação do procedimento descrito acima, o projetista deve procurar construir diagramas de interação o mais inteligíveis possível. Por exemplo, é possível utilizar notas explicativas para esclarecer algumas partes do diagrama de interação. Essas notas podem conter texto livre ou pseudocódigo para orientar a programação.

Outra estratégia que ajuda a construir um modelo de interações mais inteligível é utilizar os recursos de modularização (alt, loop, opt).



Sempre que for adequado, o projetista deve fazer, com base nos princípios de projeto, com que as classes de domínio enviem mensagens entre si. Dessa forma, o controlador envia uma mensagem para um objeto do domínio e este, por sua vez, dispara o envio de mensagens para outros objetos do domínio, o que evita que o controlador precise ter conhecimento desses últimos.

Em que momento construir o modelo de interações?

Alguns textos sobre modelagem de sistemas orientados a objetos indicam o início da modelagem de interações já na atividade de análise. Outros defendem o início de sua construção somente na etapa de projeto. O fato é que a distinção entre essas duas fases não é tão nítida na modelagem orientada a objetos e a modelagem de interações pode ser realizada em ambas as fases. Vamos entender melhor?

Análise

Nesta fase, podemos começar a construir o modelo de interações logo depois que uma primeira versão do modelo de casos de uso estiver pronta. Inicialmente, o modelo de interações pode ser utilizado para representar apenas os objetos participantes em cada caso de uso e com mensagens exibindo somente o nome da operação.

Projeto

Já na etapa de projeto, o modelo de interações deve ser mais refinado, incluindo:

- Criação e destruição de objetos.
- Detalhes sobre o tipo.
- Assinatura completa de cada mensagem etc.

Processo incremental e interativo

Nos processos incremental e interativo de desenvolvimento, cujos modelos evoluem juntos durante o desenvolvimento do sistema, eles são interdependentes. Isto ocorre mesmo que estes modelos representem visões distintas do sistema.

Os itens a seguir demonstram como o modelo de interações se relaciona aos de casos de uso e de classes.

Modelo de casos de uso ➔ modelo de interações

Utilizamos os cenários extraídos do modelo de casos de uso como fonte de identificação de mensagens na modelagem de interações. É adequado que se verifique se cada cenário relevante para todos os casos de uso foi considerado na modelagem de interações.

Modelo de classes de análise ➔ modelo de interações

Utilizamos informações provenientes do modelo de classes para construir modelos de interações. Em particular, algumas responsabilidades já podem ter sido definidas durante a modelagem de classes de análise. A notação do diagrama de interações não precisa ser totalmente utilizada na fase de análise. Em particular, podemos definir diagramas de interação que apresentam as mensagens apenas com os seus nomes. Essa definição inicial deve ser refinada posteriormente com a utilização da sintaxe mais avançada.

Modelo de interações ➔ modelo de casos de uso

A partir do conhecimento adquirido com a construção do modelo de interações, podemos aperfeiçoar e validar os cenários do modelo de casos de uso.

Modelo de interações ➔ modelo de classes

Surge da necessidade de se disponibilizar informações necessárias para a alocação e o detalhamento de operações (métodos) durante a construção de modelos de interações. Exemplo: informações que não haviam sido identificadas anteriormente, mas, necessárias a certa classe de análise. Portanto, durante a modelagem de interações, operações são identificadas e novos atributos também podem ser identificados. Outro elemento, comumente identificado ou validado pela construção do modelo de interações, é a associação, em que uma mensagem entre objetos implica a existência de uma associação entre as classes envolvidas. Além disso, também é comum identificarmos novas classes , principalmente aquelas relacionadas ao domínio da solução do problema.

Atividade 3

Qual é um dos passos iniciais para construir um diagrama de interação na UML?

A Definir a hierarquia de classes no sistema.

B Definir o caso de uso a modelar, identificando suas operações.

C Criar notas explicativas para todas as interações.

D Estabelecer os requisitos funcionais do sistema.

E Implementar pseudocódigo para todas as mensagens.

A alternativa B está correta.

O primeiro passo essencial na construção de um diagrama de interação é determinar o caso de uso que será modelado e identificar suas operações. Esse passo inicial é muito importante para compreender o contexto e as funções específicas que os objetos devem desempenhar, permitindo a correta identificação das interações e das mensagens trocadas entre eles.

Modelagem de interação na prática

A modelagem de interação na prática, fundamental para o desenvolvimento eficiente de sistemas de software, permite aos desenvolvedores e analistas uma melhor visualização e compreensão da interação entre os diferentes componentes de um sistema. Utilizando diagramas de sequência e de colaboração, é possível mapear as trocas de mensagens entre objetos e identificar claramente os fluxos de controle e dados. Esse entendimento detalhado facilita a identificação de requisitos, a detecção de possíveis problemas de integração e a otimização do design do sistema.

Vamos refletir sobre isso utilizando o minimundo locadora de veículos? Suponha que, após entrevistar o responsável, chegou-se às seguintes conclusões.

1. Cada cliente só pode alugar um veículo de cada vez, e o aluguel só pode ser feito por um funcionário acessando o sistema.
2. Um cliente pode consultar remotamente os veículos disponíveis, que são categorizados por tipo.
3. O cliente pode prorrogar remotamente o aluguel, sem necessidade de intervenção de algum funcionário.
4. A devolução do veículo também só pode ser feita por um funcionário acessando o sistema, e não precisa ser o mesmo funcionário que registrou o aluguel.
5. Existem 3 tipos de veículos: luxo, normal e popular.
6. Os veículos de um mesmo tipo têm preços e características similares.
7. Quando a locadora adquire um veículo, um funcionário deve cadastrá-lo no sistema.

Este vídeo apresenta a modelagem de interação na prática, a partir de determinado minimundo, utilizando a UML.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O caso de uso escolhido é o alugar veículo, e as operações identificadas são: identificar funcionário (passos 2 a 4), identificar cliente (passos 5 a 6), disponibilizar veículo (passos 7 a 9) e cadastrar aluguel (passos 10 e 11).

A sequência de passos do caso de uso se reflete nas mensagens entre os objetos do diagrama de sequência, e na ordem na qual são executadas. O objeto da classe de interface ITelaAluguel proporciona a comunicação com o ator; o objeto da classe de controle ControleAluguel contém a sequência dos passos das operações internas executadas para que o caso de uso seja realizado; e o objeto da classe de interface AcessoDados contém os métodos de acesso à base de dados.

Os demais objetos se referem às classes de entidade (Funcionário, Cliente, TipoVeículo, Veículo e Aluguel), que se referem aos objetos tratados no mundo real e que precisam ter seus dados associados preservados (gravados em banco).

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

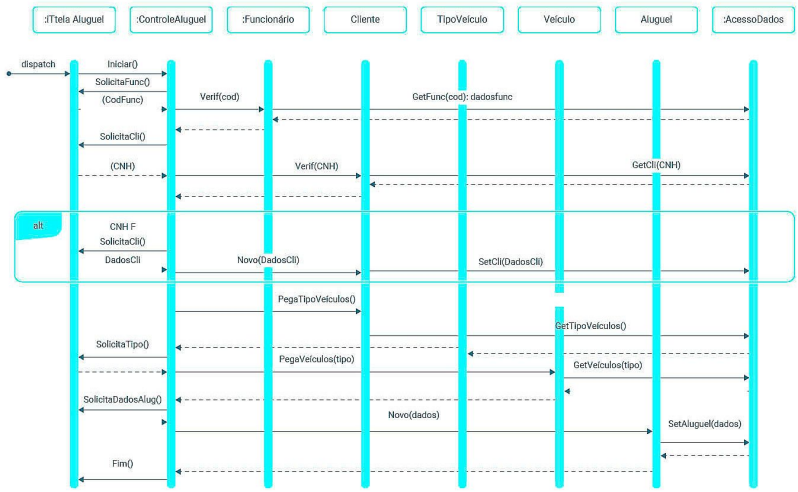


Diagrama de sequência.

Observe que toda a lógica do funcionamento do caso de uso é conduzida pelo objeto da classe de controle, após ser iniciado pela ação do ator. A partir desse momento até o encerramento do caso de uso, o objeto da classe ControleAluguel determina todas as mensagens a serem trocadas e a ordem na qual ocorrem. Ele também se comunica com todas as demais classes, chamando suas operações (métodos) para que os passos do caso de uso sejam executados.

Veja o caso de a CNH não estar na base de dados. Ocorre a solicitação dos demais dados do cliente, e seu armazenamento na base. Se a CNH estiver na base, significa que o cliente já é cadastrado e não é necessário solicitar seus dados ao ator.

Atenção

Não existe apenas um diagrama de sequência correto para esse caso de uso. A maneira de ele ser executado pode variar, o importante é que os princípios da orientação a objetos sejam seguidos.

Atividade 4

O diagrama de sequência representativo do caso de uso alugar veículo possui dois objetos de uma classe de controle, dois objetos de interface e vários objetos de entidade. Indique quais métodos você, como desenvolvedor, implementaria para que a lista de veículos de determinado tipo seja apresentada ao ator na ordem de execução:

A SocilitaTipo(), PegaVeículo(Tipo), GetVeículos(tipo), SolicitaDadosAlug().

B SocilitaTipo(), PegaTipoVeículo(), GetVeículos(tipo), SolicitaDadosAlug().

C SocilitaTipo(), PegaVeículo(Tipo), GetTipoVeículos(), SolicitaDadosAlug().

D SocilitaTipo(), GetVeículos(tipo), PegaVeículo(Tipo), SolicitaDadosAlug().

E SocilitaTipo(),GetVeículos(tipo), PegaTipoVeículo(), SolicitaDadosAlug().

A alternativa A está correta.

Os métodos realizados para que a lista de veículos seja apresentada ao ator são executados logo após o ator responder ao solicitado pelo método SolicitaTipo() do objeto da classe ControleAluguel. Ao fornecer o tipo de veículo desejado, o ControleAluguel executa o método PegaVeículo(Tipo), que inicia o método GetVeículos(tipo) do objeto TipoVeículo, que é respondido pelo objeto AcessaDados.

Diagrama de classes de projeto: atributos e métodos

Vamos tratar agora do diagrama de classes de projeto. Esse diagrama é um refinamento do diagrama de classes construído na fase de análise. Nesse refinamento, algumas classes sofrem alterações de suas propriedades com o objetivo de transformar o modelo de classes de análise no modelo de classes de projeto, assim como suas notações adicionais, transformações sobre atributos, operações e associações.

Além disso, vamos abordar também os relacionamentos entre classes, apresentando outros elementos de notação e princípios de modelagem que não são usados na análise, mas que são importantes na construção do modelo de classes de projeto.

Neste vídeo, compreenda a especificação de atributos e de métodos do diagrama de classes elaborado na etapa de projeto do processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição

O diagrama de classes de projeto apresenta os dados que serão necessários para a construção do sistema de informação. Esse é o diagrama que possui o maior número de símbolos para sua diagramação.

Comentário

O seu principal objetivo é permitir a visualização dos componentes das bases de dados do sistema, ou seja, as classes e seus objetos, relações entre elas, suas operações, seus métodos, suas interfaces e seus pacotes.

Também é papel desse diagrama atender às demandas dos casos de uso, armazenando as estruturas de dados projetadas para o sistema. Entenda melhor o foco do diagrama nas fases de análise e de projeto na comparação, a seguir:

Fase de análise	Fase do projeto
O diagrama de classes pode ser usado para produzir o modelo conceitual que representará as classes e seus atributos (não consideramos aspectos relacionados ao software). Nos interessa apenas o domínio do problema (representação das informações necessárias ao sistema, conceitos e características observados no ambiente).	Já na fase de projeto, definimos a solução do problema, ou seja, damos importância ao software por meio do qual as classes serão programadas, evidenciamos detalhes da implementação, como, por exemplo, os tipos de dados dos atributos (*varchar, integer, date* etc.) e suas operações e métodos.

Especificação de atributos

No modelo de classes de análise, os atributos são definidos somente pelo nome. No entanto, a sintaxe completa da UML para definição de um atributo é bem mais detalhada. Essa sintaxe, usada na fase de especificação do modelo de classes de projeto, é a seguinte:

```
::= [/] [visibilidade] nome[: tipo] [= valor_inicial]
```

A visibilidade de um atributo indica seu nível de acesso. Ou seja, ela permite definir quais atributos de um objeto são acessíveis por outros objetos. A visibilidade de um atributo pode ser pública, protegida, privativa ou de pacote. A primeira é o que chamamos de **default** (se nenhuma visibilidade for especificada, essa é assumida).

Default

O termo default diz respeito a um parâmetro configurado de forma automática sem intervenção do usuário, ou seja, um parâmetro utilizado como padrão pelo sistema.



Comentário

A propriedade de visibilidade serve para implementar o encapsulamento da estrutura interna da classe. Somente as propriedades realmente necessárias ao exterior da classe devem ser definidas com visibilidade pública. Todo o resto deve ser definido através das visibilidades protegida e privativa.

A seguir, temos a lista de tipos de visibilidades, seus símbolos e significados:

Pública (+)

Qualquer objeto externo pode obter acesso ao elemento, desde que tenha uma referência para a classe em que o atributo está definido.

Protegida (#)

O elemento protegido é visível para subclasses da classe em que este foi definido.

Privativa (-)

O elemento privativo é invisível externamente para a classe em que este está definido.

Pacote (~)

O elemento é visível a qualquer classe que pertence ao mesmo pacote no qual está definida a classe.

Veja a o que correspondem alguns elementos na sintaxe do atributo:

Elemento NOME: corresponde ao nome do atributo. Na verdade, somente esse elemento é obrigatório na sintaxe de declaração de um atributo, visto que os demais são opcionais.

Elemento TIPO: especifica o tipo do atributo. Esse elemento é dependente da linguagem de programação na qual a classe deve ser implementada. O tipo de um atributo pode ser definido pela utilização de um tipo primitivo da linguagem de programação a ser utilizada na implementação.



Exemplo

Se estamos utilizando a linguagem Java, temos, dentre outros, os seguintes tipos primitivos: *int, float e boolean*.

Conheça outras possibilidades de definições para os atributos:

Valor

Um valor inicial também pode ser declarado para o atributo. Dessa forma, sempre que um objeto de uma classe é instanciado, o valor inicial declarado é automaticamente definido para o atributo. Assim como o tipo, o valor inicial de um atributo também é dependente da linguagem de programação.

Classe

Pode-se definir um atributo derivado em uma classe. Um atributo é derivado quando o seu valor pode ser obtido a partir do valor de outro(s) atributo(s). Por exemplo, o valor da idade de uma pessoa pode ser obtido a partir de sua data de nascimento. Um atributo derivado é representado com uma barra inclinada à esquerda (/). Atributos derivados normalmente são definidos por questões de desempenho.

Multiplicidade

Os atributos também têm multiplicidade. Esta determina o número mínimo e máximo de valores que um atributo pode conter.

A seguir, podemos ver as multiplicidades e seus significados, que são similares às multiplicidades dos relacionamentos entre classes:

0..1: Indica que o atributo tem no mínimo zero (é opcional) e no máximo um valor.

1..1: Indica que o atributo tem no mínimo um (é obrigatório) e no máximo um valor.

0..*: Indica que o atributo tem no mínimo zero (é opcional) e no máximo muitos valores.

: Indica que o atributo tem muitos valores e equivale a 0.. (é opcional).

1..*: Indica que o atributo tem no mínimo um (é obrigatório) e no máximo muitos valores.

2..6: Indica que o atributo tem no mínimo dois e no máximo seis valores.

Especificação de operações/métodos

Na descrição do modelo de interações, declaramos que as operações de uma classe são identificadas em consequência da identificação de mensagens enviadas de um objeto a outro.



No detalhamento dessas operações, devemos considerar diversos aspectos:

- Seu nome
- Lista de parâmetros
- Tipo de cada parâmetro
- Tipo de retorno

As operações de uma classe correspondem a algum processamento realizado por essa classe. Em termos de implementação, uma operação é uma rotina associada a uma classe. A sintaxe definida na UML para uma operação é a seguinte:

```
[visibilidade] nome([parâmetros]) [: tipo-retorno] [{propriedades}]
```

A visibilidade segue a mesma simbologia das visibilidades para atributos. O elemento nome corresponde ao nome dado à operação.

Operação pública: pode ser ativada com a passagem de uma mensagem para o objeto. Se for protegida, ela só é visível para a classe e para seus descendentes.

Operação privativa: somente objetos da própria classe podem executá-la.

> Os parâmetros de uma operação correspondem às informações que ela recebe quando é executada.

Normalmente, essas informações são fornecidas pelo objeto remetente da mensagem que pede a execução da operação no objeto receptor. Uma operação pode ter zero ou mais parâmetros, que são separados por vírgulas. Cada parâmetro da lista tem a seguinte sintaxe:

```
[direção] nome-parâmetro: tipo-parâmetro
```

O elemento direção serve para definir se o parâmetro pode ou não ser modificado pela operação. O modelador pode definir se o parâmetro é de entrada (*IN*), saída (*OUT*) ou ambos (*INOUT*). Se a direção for omitida, o valor assumido pelo parâmetro será IN, de entrada.

A seguir, temos as possíveis direções na definição de um parâmetro pela UML:

IN

Parâmetro de entrada: não pode ser modificado pela operação. Serve somente como informação para o objeto receptor.

OUT

Parâmetro de saída: pode ser modificado pela operação para fornecer alguma informação ao objeto remetente.

INOUT

Parâmetro de entrada que pode ser modificado.

Veja a o que correspondem alguns elementos na sintaxe do métodos:

nome-parâmetro: corresponde ao nome do parâmetro. Cada parâmetro deve ter um nome único dentro da assinatura da operação.

tipo-parâmetro: corresponde ao tipo do parâmetro e é dependente da linguagem de programação.

tipo-retorno: representa o tipo de dados do valor retornado por uma operação. Esse tipo depende da linguagem de programação.

Atividade 1

Temos quatro tipos de especificação de visibilidade de atributos: público, protegido, privativo e pacote. A seguir, listamos definições relacionadas à visibilidade de um atributo. Assinale a alternativa que contenha a definição correta do atributo público.

A Qualquer atributo visível para classes da subclasse em que foi definido.

B Qualquer objeto externo pode obter acesso a qualquer tipo de elemento da classe.

C Qualquer objeto externo pode obter acesso ao atributo, desde que tenha uma referência para a classe externa ao pacote.

D Qualquer objeto externo pode obter acesso ao atributo, desde que tenha uma referência para a classe em que o atributo está definido.

E Qualquer objeto pode obter acesso ao elemento, desde que tenha uma referência para a classe externa deste elemento.

A alternativa D está correta.

A visibilidade (+) de um atributo público indica que as propriedades desse atributo são visíveis por qualquer objeto externo, isto é, pertencente a outra classe, contanto que tenha uma referência para a classe em que o atributo está definido.

Diagrama de classes de projeto: associações

O conhecimento teórico sobre diagramas de classes de projeto e suas associações fornece a base necessária para compreender e modelar as relações entre diferentes classes em um sistema de software. Essas associações definem como os objetos interagem e colaboram, facilitando a organização e a estruturação do código. Entender esses conceitos permite criar sistemas mais coesos e bem estruturados, melhorando a manutenção e a escalabilidade do software.

Assista ao vídeo para compreender a especificação de associações do diagrama de classes elaborado na etapa de projeto do processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Especificação de associações

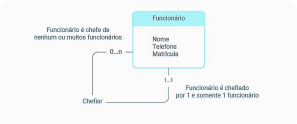
No modelo de classes de análise, relacionamentos entre objetos são normalmente definidos apenas com o uso da associação (ou como um de seus casos especiais, a generalização, a agregação ou a composição).

> As associações são os mecanismos que permitem aos objetos se comunicarem, descrevendo a conexão entre as classes.

Podem ter uma regra que especifica o propósito da associação e podem ser unidirecionais ou bidirecionais. Cada ponta da associação também possui um valor de multiplicidade que indica como os objetos de um lado se relacionam com os do outro lado. Existem diversos tipos de associação, que são modelados desde a etapa de análise:

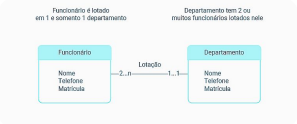
Unárias, reflexivas ou recursivas (autoassociação)

Representam relacionamentos que ocorrem entre objetos da mesma classe.



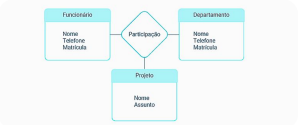
Binárias

Representam relacionamentos que ocorrem entre objetos de duas classes.



Ternárias ou n-árias

Representam relacionamentos que ocorrem entre objetos de mais de duas classes.



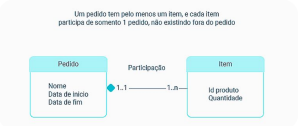
Agregação

Representa relacionamentos todo-parte em que a classe do lado parte pode existir independentemente da classe do lado todo.



Composição

Representa relacionamentos todo-parte em que a existência da classe do lado parte depende da existência da classe do lado todo.



Generalização/especialização

Representa relacionamentos entre um elemento genérico e um ou mais elementos específicos, em que o mais específico possui todas as características do elemento genérico e contém ainda particularidades não encontradas no genérico. Essa característica também é chamada de herança.



Classe associativa

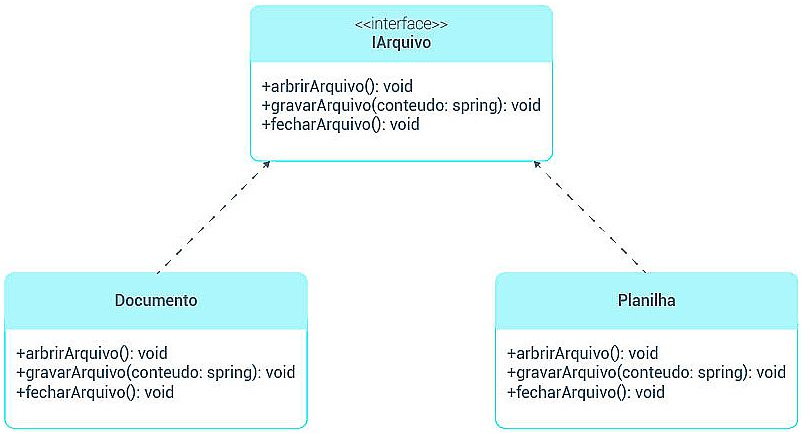
Representa relacionamentos entre classes com multiplicidade muitos para muitos e possuem propriedades próprias do relacionamento, e não das classes associadas.



Na etapa de projeto, uma espécie de associação relevante é a dependência, que representa relacionamentos entre classes, na qual uma é dependente da outra. Qualquer modificação na classe independente afetará diretamente objetos da classe dependente.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de associação de dependência.

Uma dependência entre classes indica que uma classe depende dos serviços fornecidos pela outra. No modelo de análise, é suficiente para o modelador identificar a existência de associações entre classes, que é uma forma de dependência. Mas, na fase de especificação do modelo de classes, essa dependência precisa ser mais bem definida pelo projetista, uma vez que ela tem influência na forma utilizada para implementar as classes envolvidas.

Vamos refletir sobre isso descrevendo, então, detalhes das possíveis formas de dependência? Para isso, considere duas classes, A e B, onde A depende de B. Veremos que vários tipos de dependência entre essas duas classes podem existir. Esses tipos são descritos a seguir:

- **Dependência por atributo**: A possui um atributo cujo tipo é B. A dependência por atributo é também chamada de dependência estrutural.
- **Dependência por variável global**: A utiliza uma variável global cujo tipo é B.
- **Dependência por variável local**: A possui alguma operação cuja implementação utiliza uma variável local de tipo B.
- **Dependência por parâmetro**: A possui pelo menos uma operação, que possui pelo menos um parâmetro, cujo tipo é B.

A notação da UML para representar uma dependência no diagrama de classes é de uma seta tracejada ligando as classes envolvidas. A seta sai da classe dependente e chega na classe da qual depende. Ainda com respeito à notação, as dependências podem ser estereotipadas para indicar o tipo de dependência existente entre duas classes. Os estereótipos da UML para rotular relacionamentos de dependência são:

<>

Para variável global.

<>

Para variável local.

<>

Para variável local.

Atividade 2

Assinale a alternativa que apresenta os dois tipos de associações que correspondem, respectivamente, aos seguintes relacionamentos: a existência da classe do lado parte depende da existência da classe do lado

todo; duas ou mais classes herdam características de uma superclasse, além de possuírem suas próprias características.

A Associação unária e associação binária.

B Associação binária e agregação.

C Agregação e generalização.

D Composição e generalização.

E Generalização e composição.

A alternativa D está correta.

A composição representa um relacionamento no qual a existência da classe do lado parte depende diretamente da existência da classe do lado todo. Se a classe do lado todo for destruída, a classe do lado parte também será destruída. Já a generalização representa um relacionamento em que duas ou mais classes herdam características de uma superclasse. As subclasses possuem todas as características da superclasse, além de suas próprias características específicas.

Transformação e navegabilidade de associações

Permite entender como as classes interagem dentro de um sistema. A navegabilidade define a direção na qual a associação pode ser percorrida, influenciando a acessibilidade dos dados. Já a transformação de associações abrange a conversão de relacionamentos conceituais em implementações práticas, garantindo a integridade e a funcionalidade do sistema.

Compreender esses conceitos é importante para projetar sistemas coesos, eficientes e de fácil manutenção.

Este vídeo explica como transformar associações conceituais em implementações práticas no diagrama de classes, abrangendo a conversão dos relacionamentos e a navegabilidade, que determina a direção em que a associação pode ser explorada, no contexto do projeto de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Transformação de associações em dependências

No modelo de classes de análise, o modelador especifica uma ou mais associações entre uma classe e as demais apresentadas. Na passagem do modelo de classes de análise para o de projeto, o modelador deve estudar cada associação para identificar se ela pode ser transformada em dependências. A razão para essa transformação é definir melhor o encapsulamento.

Comentário

Quanto menos dependências estruturais houver no modelo de classes, maior será a qualidade do projeto (do ponto de vista do encapsulamento e do acoplamento das classes constituintes). De um modo geral, as associações que ligam classes de entidade permanecem como associações no modelo de projeto.

Classes persistentes e transientes

As classes são divididas em:

Persistentes	Não persistentes (transientes)
Presumem que seus objetos precisam, de alguma maneira, ser preservados mesmo após o encerramento da utilização do sistema.	Terão seus objetos destruídos durante ou na finalização do uso do sistema. Estas são diferenciadas por estereótipos <<transient>>.

Classes de interface

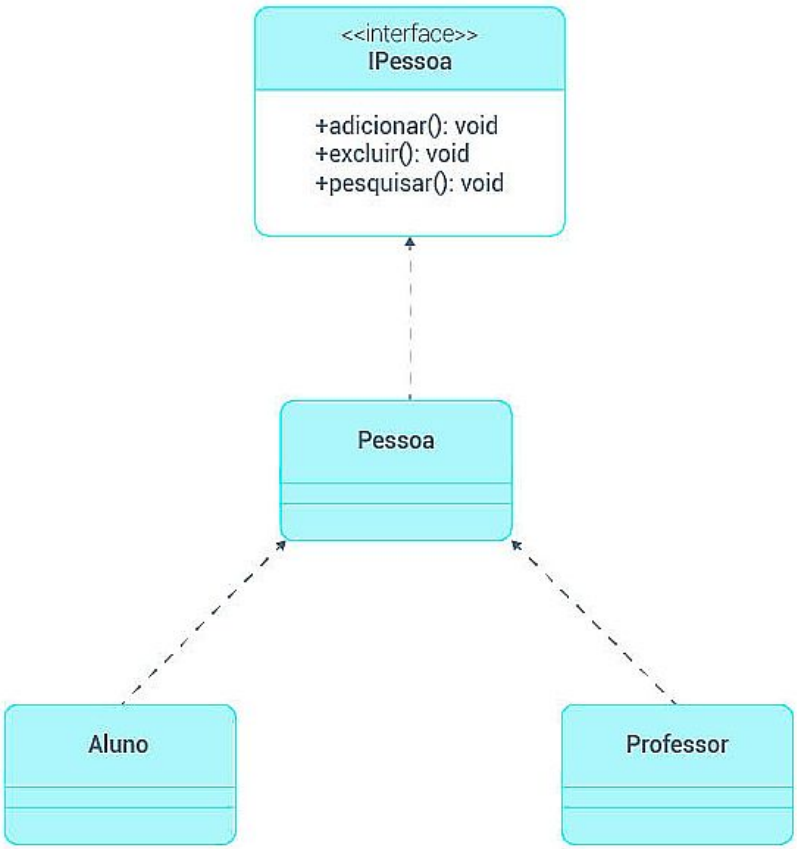
Uma classe de interface é uma coleção de operações com um nome e é utilizada para especificar um tipo de serviço sem especificar como será sua implementação. Essas classes não têm métodos concretos, apenas o declaram para que outra classe possa implementá-lo.

> Classes de interface possibilitam que objetos externos ao sistema colaborem com uma ou mais classes do sistema.

São exemplos as interfaces gráficas, as de interações entre os usuários e as telas do sistema. Há um estereótipo para ela () e sua implementação utiliza uma reta tracejada oriunda da classe de implementação, contendo a seta vazia na extremidade que aponta para a classe de interface, como ilustra o exemplo a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de classe de interface.

Navegabilidade de associações

As associações podem ser classificadas em bidirecionais e unidirecionais. Uma associação bidirecional indica que há um conhecimento mútuo entre os objetos associados. Ou seja, se um diagrama de classes exibe uma associação entre duas classes C1 e C2, as duas assertivas a seguir são verdadeiras:

- Cada objeto de C1 conhece todos os objetos de C2 aos quais está associado.
- Cada objeto de C2 conhece todos os objetos de C1 aos quais está associado.

Graficamente, uma associação unidirecional é representada adicionando-se um sentido à associação. A classe para a qual o sentido aponta é aquela cujos objetos não possuem a visibilidade dos objetos da outra classe.

Durante a construção do modelo de classes de análise, associações são normalmente consideradas “navegáveis” em ambos os sentidos, ou seja, as associações são bidirecionais. No modelo de classes de projeto, o modelador deve refinar a navegabilidade de todas as associações.

Atenção

Pode ser que algumas associações precisem permanecer bidirecionais. No entanto, se não houver essa necessidade, recomenda-se transformar a associação em unidirecional. Isso porque uma associação bidirecional é mais difícil de implementar e de manter do que uma associação unidirecional correspondente.

Para entender isso, basta ver que o acoplamento entre as classes envolvidas na associação é maior quando a navegabilidade é bidirecional, prejudicando a modularização. Portanto, um dos objetivos a alcançar durante o projeto é identificar quais navegabilidades são realmente necessárias.

A definição da navegabilidade se dá em função da troca de mensagens ocorridas nas interações entre objetos do sistema. Mais especificamente, devemos analisar os fluxos das mensagens entre objetos no diagrama de interações. Dados dois objetos associados, A e B, se há pelo menos uma mensagem de A para B em alguma interação, a associação deve ser navegável de A para B. Da mesma forma, se existir pelo menos uma mensagem de B para A, então a associação deve ser navegável de B para A.



A definição do sentido da navegabilidade é feita em função das mensagens identificadas na modelagem de interação. Em particular, se um objeto do tipo A envia uma mensagem para outro do tipo B, então deve haver uma navegabilidade no sentido de A para B no diagrama de classes.

Mesmo em situações em que a navegabilidade de uma associação precise ser bidirecional, é possível implementar apenas um de seus sentidos.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de navegabilidade.

Atividade 3

O entendimento da transformação e da navegabilidade de associações é importante para saber como se dá a interação de classes em um sistema. Assinale a alternativa que melhor descreve a transformação de associações em dependências e a navegabilidade de associações:

A Transformar associações em dependências aumenta o acoplamento entre classes, e a navegabilidade unidirecional é mais difícil de implementar.

B Transformar associações em dependências melhora o encapsulamento, e a navegabilidade bidirecional reduz o acoplamento entre classes.

C Transformar associações em dependências melhora o encapsulamento, e a navegabilidade unidirecional reduz o acoplamento entre classes.

D Transformar associações em dependências reduz a modularidade, e a navegabilidade bidirecional é mais fácil de manter.

E Transformar associações em dependências reduz o encapsulamento, e a navegabilidade unidirecional aumenta o acoplamento entre classes.

A alternativa C está correta.

Transformar associações em dependências ajuda a melhorar o encapsulamento das classes, pois diminui as dependências estruturais entre elas, resultando em um design de software mais robusto e modular. Além disso, a navegabilidade unidirecional é preferível quando há possibilidade, pois reduz o acoplamento entre classes, facilitando a manutenção e a implementação do sistema. Uma associação bidirecional, por outro lado, aumenta o acoplamento e a complexidade, tornando o sistema mais difícil de manter.

Diagrama de classes de projeto na prática

No desenvolvimento de sistemas, o diagrama de classes de projeto tem o papel de detalhar a estrutura do software. Esse diagrama especifica:

- As classes que compõem o sistema.
- Os atributos.
- Os métodos.
- As associações entre classes.

Na prática, construir um diagrama de classes de projeto é fundamental para garantir um design claro e bem estruturado, facilitando a comunicação entre os membros da equipe de desenvolvimento e servindo como base para a implementação do código. Portanto, dominar a criação e interpretação de diagramas de classes de projeto é essencial para qualquer desenvolvedor que almeja construir sistemas robustos e escaláveis.

Vamos construir um diagrama de classes sobre o minimundo locadora de veículos, mais especificamente relacionado ao caso de uso alugar veículo, o qual teve seu diagrama de sequência desenvolvido anteriormente?

Comentário

As informações (métodos, atributos e associações) do diagrama são referentes ao caso de uso alugar veículo. Os métodos definidos nesse diagrama de sequência serão utilizados no diagrama de classes de projeto. A confecção dos diagramas de sequência dos demais casos de uso possivelmente trarão novos elementos de informação para o diagrama de classes de projeto do minimundo considerado.

Este vídeo apresenta a prática da elaboração do diagrama de classes de projeto, a partir do refinamento do diagrama de classes gerado na etapa de análise do processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Vejamos o diagrama de classes elaborado na etapa de análise:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

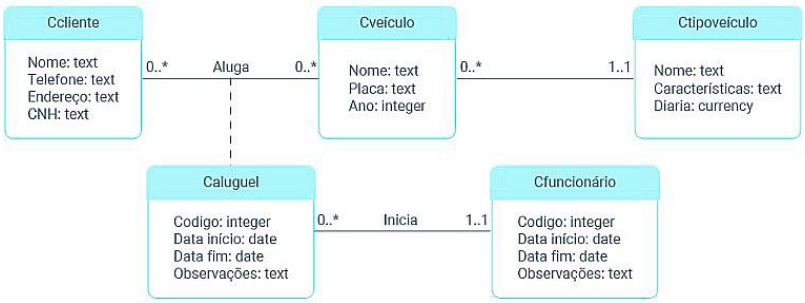


Diagrama de classes de projeto (parcial).

Com base no diagrama de classes de análise e no diagrama de sequência ilustrado na imagem a seguir, será elaborado o diagrama de classes de projeto, que representa um refinamento do diagrama de classes de análise.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

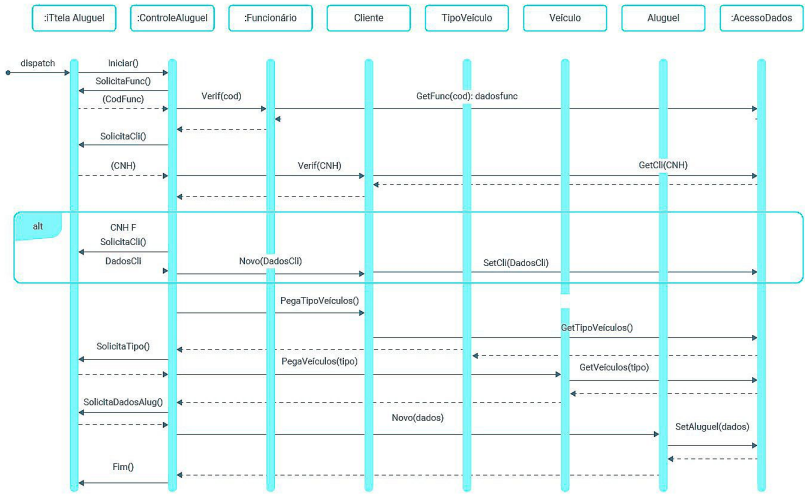


Diagrama de sequência.

No diagrama do projeto, os atributos das classes, os métodos e as associações são adequados às questões de implementação, ficando mais detalhado do que o diagrama de classes de análise original, o qual só se preocupava com as questões conceituais do mundo real.

Por questões de simplicidade, só serão representados alguns atributos e métodos, ficando claro que os atributos e métodos não contemplados devem seguir os mesmos conceitos. Vejamos o diagrama do projeto proposto como solução:

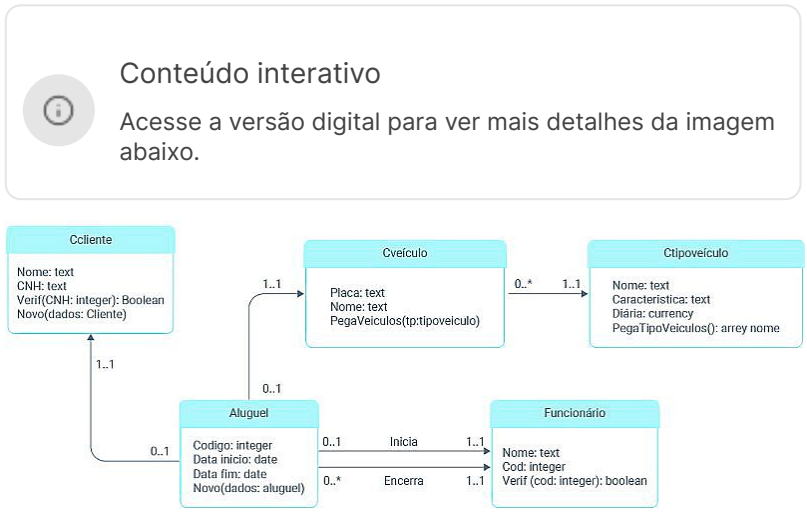


Diagrama de classes de projeto (parcial).

Veja que todos os métodos identificados no diagrama de sequência desenvolvido estão representados no diagrama de classes do projeto, bem como a navegabilidade de cada associação. Vale dizer que todos os atributos são privados (na frente do atributo). Dessa forma, não é possível alterar diretamente qualquer atributo (encapsulamento). Vale destacar mais alguns pontos importantes:

Métodos Set e Get

As classes terão métodos Set e Get, que alteram e leem os valores dos atributos privados, porém não são representados no diagrama de classes de projeto, pois são subentendidos.

Ferramenta CASE

Qualquer ferramenta CASE (Computer-aided software engineering) gera os métodos Set e Get para os atributos privados.

Responsabilidade do projetista

Cabe ao projetista definir se alguma verificação deve ser incluída no código desses métodos (por exemplo, validação de datas).

Se uma ferramenta CASE for utilizada para geração automática de código, os tipos detalhados são necessários, mas se o diagrama for usado exclusivamente por desenvolvedores de software, o excesso de informação pode poluir o diagrama e dificultar seu entendimento. No diagrama de sequência, por exemplo, não há troca de mensagens entre as classes de entidade.



Dica

Um diagrama de sequência para o mesmo minimundo poderia prever tais mensagens, e não estaria errado. Como sabemos, não há apenas uma forma correta de modelagem.

Atividade 4

Considerando que você é um engenheiro de software realizando a modelagem de classes de projeto, qual alternativa descreve corretamente algo que ocorre na construção de diagramas de classes de projeto utilizando diagramas de interação e diagrama de classes de análise.

A Os métodos representados no diagrama de classes do projeto nem sempre são identificados no diagrama de interação.

B Os atributos representados no diagrama de classes de projeto são públicos para permitir fácil acesso e modificação.

C Os métodos Set e Get são sempre representados no diagrama de classes de projeto para garantir o encapsulamento.

D As ferramentas CASE geram automaticamente os métodos Set e Get para os atributos privados no diagrama de classes de projeto.

E Um diagrama de sequência correto nunca prevê trocas de mensagens entre classes de entidade.

A alternativa D está correta.

Qualquer ferramenta CASE gera automaticamente os métodos Set e Get para os atributos privados, e esses métodos não precisam ser representados no diagrama de classes de projeto, pois são subentendidos. Isso é uma prática comum para garantir o encapsulamento e manter o diagrama de classes mais limpo e fácil de entender.

Diagrama de estado

Permite modelar o comportamento dinâmico e o fluxo de controle dentro de uma aplicação. Diagramas de estado ajudam a entender as transições entre diferentes estados de um objeto ao longo do tempo, enquanto diagramas de atividades detalham o fluxo de trabalho e processos paralelos. Dominar esses conceitos possibilita a criação de sistemas mais eficientes e bem estruturados, facilitando a comunicação entre os desenvolvedores e melhorando a qualidade e a manutenção do software.

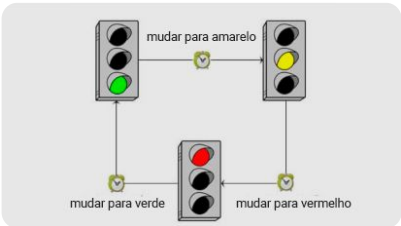
Neste vídeo, compreenda a aplicação do diagrama de transição de estados, utilizando a UML no processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Transição entre estados

Objetos de um sistema de software orientado a objetos se encontram em um estado particular a cada momento. Um objeto muda de estado quando acontece algum evento interno ou externo ao sistema. Quando um objeto muda de um estado para outro, diz-se que ele realizou uma **transição entre estados**. Os estados e as transições de estado de um objeto constituem o seu ciclo de vida.



Exemplo de transição de estado.

No momento de sua transição de um estado para outro, um objeto normalmente realiza determinadas ações dentro do sistema. Veja alguns pontos importantes sobre a transição dos objetos:

- Cada objeto pode passar por um número finito de estados durante a sua vida. Quando um objeto transita de um estado para outro, significa que o sistema no qual ele está inserido também está mudando de estado.
- Pela análise das transições entre estados dos objetos de um sistema de software, é possível prever todas as possíveis operações realizadas, em função de eventos que podem ocorrer.
- O diagrama da UML utilizado para realizar essa análise é o diagrama de estados, também conhecido como diagrama de máquina de estado ou de transição de estado (DTE).
- Esse diagrama permite descrever o ciclo de vida de objetos de uma classe, os eventos que causam a transição de um estado para outro e a realização de operações resultantes.

Atenção

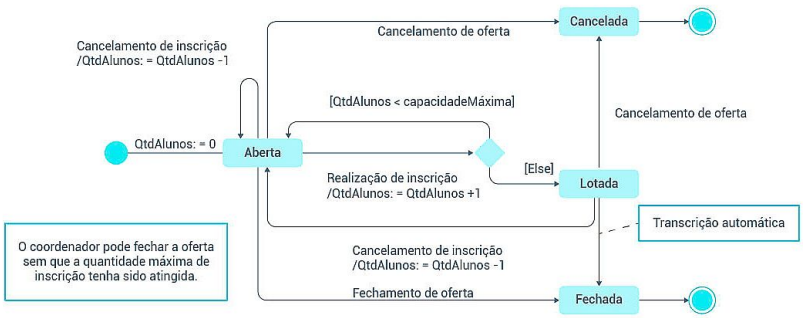
Assim como diagramas de interação não são construídos para todos os casos de uso, os diagramas de transição de estado não devem ser construídos para todas as classes de um sistema, mas apenas para aquelas que possuem um número definido de estados conhecidos, e quando o comportamento das classes de objetos é afetado e modificado pelos diferentes estados. Nessas classes, um DTE pode ser utilizado para enfatizar os eventos que resultam em mudanças de estado.

Diagrama de transição de estados

A UML tem um conjunto rico de notações para desenhar um DTE. Os elementos básicos de um diagrama de transição de estados são os estados e as transições. Associados a estas últimas estão os conceitos de **evento, ação e atividade**. Um DTE pode também conter elementos menos utilizados, mas às vezes úteis, como **transições internas, estados aninhados e estados concorrentes**. A imgem a seguir mostra um diagrama de estados para o objeto oferta de disciplina, que pode ter quatro estados: **aberta, cancelada, lotada e fechada**.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de estados.

Um estado é uma situação na vida de um objeto durante a qual ele satisfaz alguma condição ou realiza alguma atividade. Cada estado de um objeto é normalmente determinado pelos valores dos seus atributos e (ou) pelas suas ligações com outros objetos.

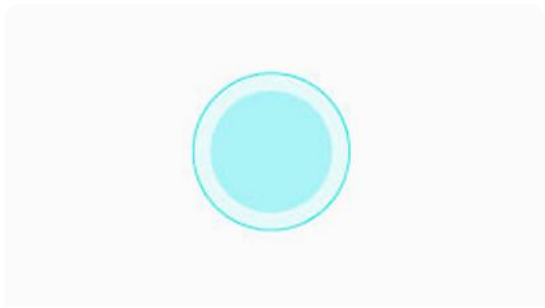
A UML utiliza as seguintes notações para representar os estados:

Estado

O retângulo com as bordas arredondadas representa um estado. Em sua forma mais simples, o retângulo de um estado possui um único compartimento.

Estado inicial

O círculo preenchido indica o estado de um objeto quando ele é criado. Só pode haver um estado inicial em um DTE, restrição que serve para definir a partir de que ponto um DTE deve começar a ser lido.



Estado final

Representado por um círculo parcialmente preenchido. Diferentemente do estado inicial, pode haver vários estados finais, dependendo dos estados terminais do objeto modelado.

Transições

Os estados estão associados a outros pelas transições. Uma transição é mostrada como uma linha conectando estados, com uma seta apontando do estado origem para o estado destino. Quando ocorre uma transição entre estados, diz-se que a transição foi disparada. Note que, em uma transição, o estado subsequente pode ser igual ao estado original. Uma transição pode ser rotulada com uma expressão:

```
evento (lista-parâmetros) [guarda] / [ação]
```

A seguir, vamos entender os elementos envolvidos nas transições entre estados:

Evento

Um evento é algo que acontece em algum ponto no tempo e que pode modificar o estado de um objeto, provocando a transição de estado. Um evento pode conter uma lista de parâmetros que são passados para fornecer informações úteis ao objeto receptor de evento. A lista de parâmetros é especificada entre parênteses. Na imagem **exemplo de diagrama de estados**, *realização de inscrição / qtdAlunos := qtdAlunos + 1* é um exemplo de evento.
Os eventos relevantes a uma classe de objetos podem ser classificados em quatro tipos:

- **Evento de chamada**: recebimento de uma mensagem de outro objeto. Pode-se pensar nesse tipo de evento como uma solicitação de serviço de um objeto a outro.
- **Evento de sinal**: recebimento de um sinal. Este é um tipo especial do evento anterior. Nesse evento, o objeto recebe um sinal de outro objeto que pode fazê-lo mudar de estado. A diferença básica entre o evento de sinal e o evento de chamada é que neste último o objeto que envia a mensagem fica esperando a execução dela. No evento de sinal, o objeto remetente continua o seu processamento após ter enviado o sinal. O evento de sinal raramente é utilizado.
- **Evento temporal**: passagem de um intervalo de tempo predefinido. Em vez de receber uma mensagem específica, um objeto pode interpretar a passagem de um certo intervalo como sendo um evento.
- **Evento de mudança**: uma condição que se torna verdadeira. O fato de determinada condição se tornar verdadeira também pode ser visto como um evento.

Condição de guarda

Uma transição pode ter também uma condição de guarda. Uma condição de guarda, ou simplesmente guarda, é uma expressão de valor lógico, da mesma forma que a utilizada para diagramas de interação. A condição de guarda de uma transição pode ser definida utilizando-se parâmetros passados no evento e atributos e referências a ligações da classe em questão. Além disso, uma condição também pode testar o valor de um estado. Uma transição na qual foi definida uma condição de guarda é disparada somente se o evento associado ocorre e a condição de guarda é verdadeira. Se uma transição não tiver uma condição de guarda, ela sempre será disparada quando o evento ocorrer. É importante não confundir a condição de guarda de uma transição com um evento de mudança (que também é definido com uma expressão de valor lógico). A expressão condicional de uma condição de guarda é sempre apresentada entre colchetes, como em *[qtdAlunos < capacidadeMáxima] e [else]*, visto no **exemplo de diagrama de estados**.

Ações

Ao transitar de um estado para outro, um objeto pode realizar uma ou mais ações. Uma ação é uma expressão que pode ser definida em termo dos atributos, das operações ou das associações da classe. Os parâmetros do evento também podem ser utilizados. Uma ação pode corresponder igualmente à execução de uma operação. A ação é representada na linha da transição e deve ser precedida por uma barra inclinada para a direita (símbolo "/"). Note que a ação associada a uma transição é executada somente se a transição for disparada. No **exemplo de diagrama de estados**, são exemplos de ações: / *qtdAlunos := qtdAlunos + 1 e / qtdAlunos := qtdAlunos – 1*.

Atividades

Semelhante a uma ação, uma atividade é algo que é executado pelo objeto. No entanto, uma atividade pode ser interrompida (considera-se que o tempo de execução de uma ação é tão insignificante que ela não pode ser interrompida). Por exemplo, enquanto a atividade estiver em execução, pode acontecer um evento que a interrompa e cause uma mudança de estado. Outra diferença entre ação e atividade é que uma atividade sempre está associada a um estado (ao contrário, uma ação está associada a uma transição).

Ponto de junção

Em algumas situações, o próximo estado de um objeto varia de acordo com o valor da condição de guarda. Se o valor da condição de guarda for verdadeiro, o objeto vai para um estado E1; se for falso, o objeto vai para outro estado E2. É como se a transição tivesse bifurcações e cada transição de saída da bifurcação tivesse uma condição de guarda. Essa situação pode ser representada em um DTE por um ponto de junção. Um ponto de junção é desenhado como um losango em que chegam uma ou mais transições (provenientes de estados diferentes) e de onde partem uma ou mais transições. A cada transição de saída do ponto de junção está associada uma condição de guarda. A transição que o objeto segue é aquela para a qual a condição de guarda é verdadeira. Note que há um ponto de junção no **exemplo de diagrama de estados**.

Estados aninhados

Podem existir estados aninhados dentro de outro estado. Um estado que contém diversos outros é dito composto. Todos os estados dentro de um estado composto herdam qualquer transição deste último. Isso significa que o uso de estados compostos geralmente torna um DTE mais legível.

Estados concorrentes

Um estado concorrente é um tipo especial de estado composto. Um objeto em um estado concorrente pode, na verdade, se encontrar em dois ou mais estados independentes.

Identificação dos elementos de um diagrama de estados

- Um professor está **licenciado** quando não está ministrando curso algum durante o semestre.
- Uma oferta de disciplina está **aberta** quando a capacidade máxima não for alcançada; caso contrário, a oferta estará **lotada**.
- Um pedido está **atendido** quando todos os seus itens estão entregues.

Conheça alguns pontos de partida para identificar estados de um objeto:

Análise dos possíveis valores de seus atributos e as ligações que ele pode realizar com outros objetos.

No entanto, a existência de atributos ou ligações não é suficiente para justificar a criação de um DTE para uma classe. O comportamento de objetos dessa classe deve depender de tais atributos ou ligações. Em relação a transições, devemos identificar os eventos que podem gerá-las. Além disso, é preciso examinar também se há algum fator que condicione o disparo da transição. Se existir, esse fator deve ser modelado como uma condição de guarda da transição. Já que as transições dependem de eventos para ocorrer, devem-se identificar esses eventos primeiramente.

Descrição dos casos de uso

Este é outro bom ponto de partida! Os eventos encontrados na descrição dos casos de uso são externos ao sistema. Contudo, uma transição pode também ser disparada por um evento interno ao sistema. Para

identificar os eventos internos relevantes a um objeto, analise os **diagramas de interação**. Esses diagramas contêm mensagens sendo trocadas entre objetos e mensagens, que nada mais são do que solicitações de um objeto a outro. Essas solicitações podem ser vistas como eventos. De uma forma geral, cada operação com visibilidade pública de uma classe pode ser vista como um evento em potencial.



Dica

Outra fonte para identificação de eventos relevantes é analisar as regras de negócio definidas para o sistema. Normalmente, essas regras possuem informações sobre condições limites que permitem identificar estados e transições.

Atividade 1

Um dos diagramas apresentados é o diagrama de transição de estados. Assinale abaixo a alternativa que define corretamente o que é um estado:

A História de vida de um objeto.

B Descrição das atividades realizadas por um objeto.

C Situações e/ou condições na vida de um objeto.

D Cenário onde está inserid um objeto.

E Elemento que define o objeto.



A alternativa C está correta.

Um objeto pode mudar de estado ao longo de seu ciclo de vida, o qual é caracterizado por situações ou condições resultantes da ocorrência de eventos.

Construção de diagramas de transições de estados

É muito importante para o desenvolvimento de sistemas, pois esses diagramas representam como os objetos respondem a diferentes eventos ao longo do tempo. Compreender esses conceitos permite aos desenvolvedores visualizar e modelar o comportamento dinâmico do sistema, identificar estados críticos e transições, e garantir que todas as possíveis situações e respostas sejam contempladas. Isso resulta em um

projeto mais robusto e previsível, facilitando a detecção de erros e a implementação de funcionalidades complexas de forma clara e estruturada.

Este vídeo apresenta a construção de diagramas de transições de estados, utilizando a UML ao longo do processo de desenvolvimento de software.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Para sistemas bastante simples, a definição dos estados de todos os objetos não é tão trabalhosa. No entanto, a quantidade de estados possíveis de todos os objetos de um sistema complexo é grande. Isso torna a construção de um DTE para o sistema inteiro uma tarefa bastante complexa.

> Para resolver o problema da explosão exponencial de estados, os diagramas de estados são desenhados por classe.

Geralmente, cada classe é simples o suficiente para que o diagrama de estados correspondente seja compreensível. Note, contudo, que essa solução de dividir a modelagem de estados por classes do sistema tem a desvantagem de dificultar a visualização do estado do sistema como um todo. Essa desvantagem é parcialmente compensada pela construção de diagramas de interação.

Nem todas as classes de um sistema precisam de um DTE. Um diagrama de estados é desenhado somente para classes que exibem um comportamento dinâmico relevante. A relevância do comportamento de uma classe depende muito de cada situação em particular. No entanto, objetos cujo histórico precisa ser rastreado pelo sistema são típicos para se construir um diagrama de estados. Uma vez selecionadas as classes para cada uma das quais se deve construir um diagrama de estados, acompanhe essa sequência de passos:

1. Identifique os estados relevantes para a classe.
2. Identifique os eventos relevantes aos objetos de uma classe. Para cada evento, identifique qual transição que ele ocasiona.
3. Para cada estado, identifique as transições possíveis quando um evento relevante ocorre.
4. Para cada estado, identifique os eventos internos e as ações correspondentes relevantes.
5. Para cada transição, verifique se há fatores que influenciam o seu disparo. Se esse for o caso, uma condição de guarda deve ser definida.
6. Verifique também se alguma ação deve ser executada quando uma transição é disparada (ações).
7. Para cada condição de guarda e para cada ação, identifique os atributos e as ligações que estão envolvidos. Pode ser que esses atributos ou ligações ainda não existam. Nesse caso, eles devem ser adicionados ao modelo de classes.
8. Defina o estado inicial e os eventuais estados finais.
9. Desenhe o diagrama de estados. Procure posicionar os estados de tal forma que o ciclo de vida do objeto possa ser visualizado de cima para baixo e da esquerda para a direita.

A construção de diagramas de estados de um sistema frequentemente leva à descoberta de novos atributos para uma classe, principalmente atributos que servem de abstrações para estados. Além disso, esse processo de construção permite identificar novas operações na classe, pois os objetos precisam reagir aos eventos que eles recebem, e essas reações são realizadas mediante operações.

Atividade 2

Depois de selecionar as classes, há uma sequência de passos a serem seguidos para construir um diagrama de estados. Quais são os passos corretos para a construção de um diagrama de transições de estados (DTE) para uma classe que exibe comportamento dinâmico relevante?

A Identificar estados, eventos, transições, condições de guarda e ações; desenhar o diagrama.

B Identificar estados, eventos, transições, ações, atributos; desenhar o diagrama.

C Identificar estados, eventos, transições, condições de guarda, ações, atributos; definir estados iniciais e finais; desenhar o diagrama.

D Identificar estados, eventos, transições, atributos; definir estados iniciais e finais; desenhar o diagrama.

E Identificar estados, eventos, transições, atributos; desenhar o diagrama.

A alternativa C está correta.

A construção de um DTE envolve uma sequência estruturada de passos: identificar estados e eventos relevantes, definir transições e condições de guarda, listar ações correspondentes, verificar e adicionar atributos ou ligações necessárias, definir estados iniciais e finais e, finalmente, desenhar o diagrama. Essa abordagem detalhada garante que todas as dinâmicas de comportamento da classe sejam corretamente representadas no diagrama.

Modelagem de estados na prática

Faz muito sentido quando uma classe apresenta alguns estados que mudam a forma dos objetos correspondentes se comportarem. A prática dessa modelagem é fundamental para a compreensão dos possíveis comportamentos dos objetos de uma classe. Para essa modelagem, no minimundo de exemplo, vamos inserir dois novos casos de uso, os quais incorporam a questão do defeito: enviar veículo para conserto e receber veículo de conserto.

Quando um veículo apresenta um defeito estando alugado, o aluguel é encerrado. Por isso, o relacionamento extend entre os casos de uso enviar veículo para conserto e encerrar aluguel. Caso o cliente queira, um novo aluguel é feito com outro carro, mas de forma independente do aluguel antigo.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

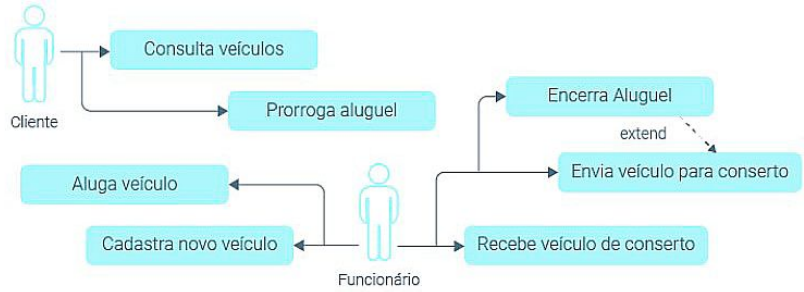


Diagrama de casos de uso.

Essas novas funcionalidades induzem à criação da classe Oficina, pois a locadora se relaciona com 3 oficinas para a realização dos consertos. Também é necessária a classe cConserto, uma classe associativa entre cVeículo e cOficina. O novo diagrama de classes de análise fica desta forma:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

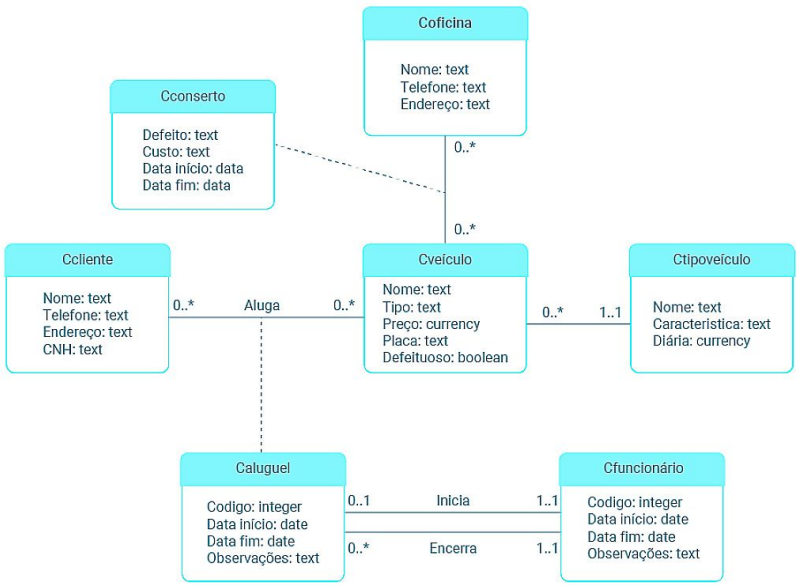


Diagrama de classes.

Esse novo requisito (conserto do veículo) é utilizado na elaboração do diagrama de estados da classe Veículo, pois a classe passa a ter estados significativos, o que justifica a criação desse tipo de diagrama.

Prática de construção de diagrama de estado

Este vídeo apresenta a prática da construção de um diagrama de estado, a partir de determinado minimundo, utilizando a UML.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Vamos modelar os estados da classe veículo. O entendimento do minimundo permite inferir que essa classe apresenta alguns estados, tais como: disponível, alugado etc. A modelagem de estados permite uma melhor compreensão de como os objetos dessa classe funcionam em cada estado.

Primeiramente, identifique os eventos relevantes aos objetos de uma classe. Exemplo: alugar e encerrar aluguel, apresentar defeito, enviar para oficina e finalizar conserto. Para cada evento, identifique qual transição ele ocasiona. No exemplo, o evento alugar muda o estado do veículo de disponível para alugado; o evento encerrar aluguel muda de alugado para disponível. O evento apresentar defeito muda o estado disponível ou alugado para indisponível. O evento enviar oficina muda o estado de indisponível para consertando. Por fim, o evento finalizar conserto muda o estado consertando para disponível. Os estados possíveis da classe Veículo são, então: disponível, alugado, indisponível e consertando.

A construção do diagrama de estado da classe permite visualizar quais operações são possíveis em cada estado. Lembre-se de que um estado é caracterizado por valores em determinados atributos, pela situação das associações com outros objetos e pelas operações permitidas. Assim, é provável que durante a modelagem de estados o diagrama de classes precise ser atualizado, incorporando novos atributos ou relacionamentos.

O diagrama de estados deve ser desenhado de tal forma que o ciclo de vida do objeto possa ser visualizado de cima para baixo e da esquerda para a direita. Veja uma solução na seguinte imagem.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

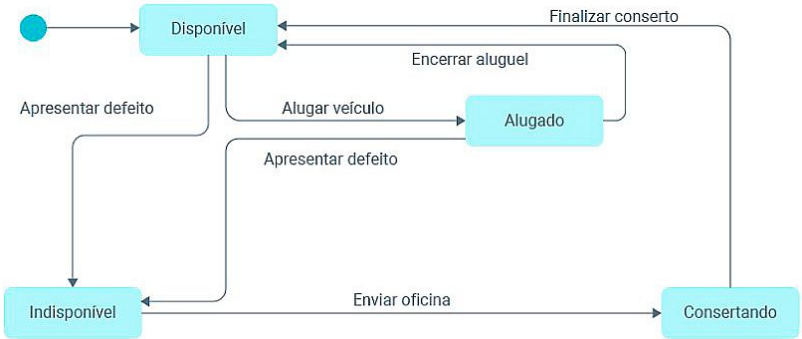


Diagrama de estados.

Atividade 3

Imagine que você seja um engenheiro de software trabalhando na modelagem de estados, qual das seguintes situações você verifica na modelagem de estados da classe veículo?

A O evento finalizar conserto muda o estado do veículo de indisponível para disponível.

B O evento apresentar defeito pode mudar o estado do veículo de alugado para consertando.

C Existem dois estados que possibilitam eventos que tornam o veículo indisponível.

D O veículo pode ser enviado para a oficina caso esteja alugado.

E O evento enviar oficina muda o estado do veículo de alugado para indisponível.

A alternativa C está correta.

A opção C é correta porque o diagrama mostra claramente que o evento apresentar defeito muda o estado do veículo de disponível para indisponível ou de alugado para indisponível.

Diagrama de atividades

É fundamental para a modelagem eficiente de processos e fluxos de trabalho em sistemas complexos. Esses diagramas oferecem uma representação visual clara das sequências de ações e decisões dentro de um sistema, facilitando a compreensão e comunicação entre desenvolvedores e partes interessadas.

Ao identificar corretamente atividades, transições e condições de guarda, todos os aspectos dinâmicos do processo são capturados com precisão. Isso contribui para projetos mais coesos e bem estruturados, essenciais para o desenvolvimento de software de alta qualidade.

Neste vídeo, compreenda a aplicação do diagrama de atividades, utilizando a UML, ao longo do processo de desenvolvimento de software. O vídeo vai abordar fluxo de controle sequencial, fluxo de controle paralelo, raias e modelagem da lógica de uma operação complexa.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

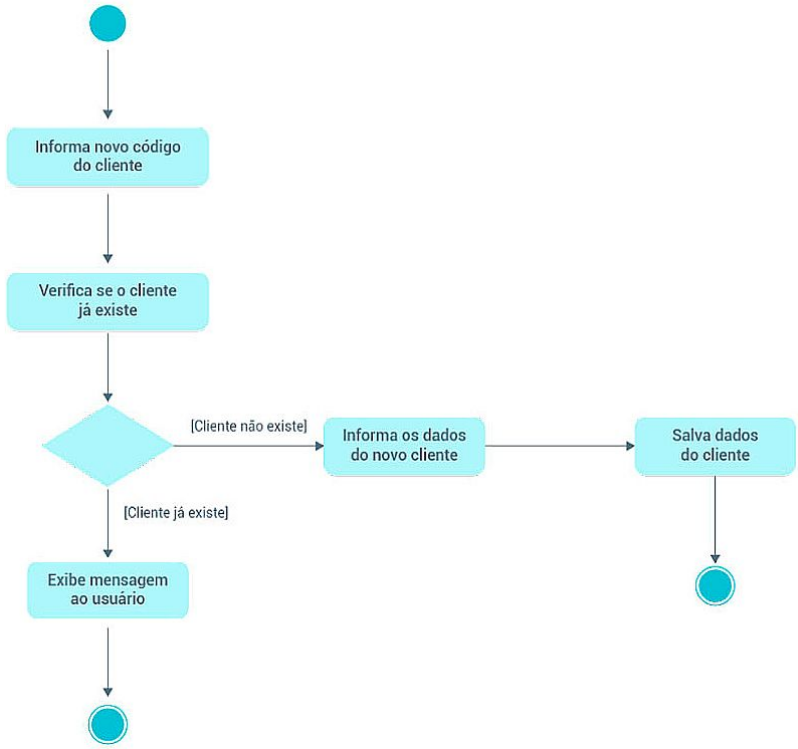
Definição

Um diagrama de atividade pode ser considerado um tipo especial de diagrama de estados, em que são representados os estados de uma atividade em vez dos estados de um objeto. Ao contrário dos diagramas de estados, que são orientados a eventos, diagramas de atividade são orientados a fluxos de controle.

Muitos autores dizem que o diagrama de atividades é um fluxograma. Na verdade, o diagrama de atividade pode ser visto como uma extensão dos fluxogramas. Além de possuir toda a semântica existente em um fluxograma (com notação ligeiramente diferente), o diagrama de atividade possui notação para representar ações concorrentes (paralelas), juntamente com a sua sincronização.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de atividades.

Os elementos de um diagrama de atividade podem ser divididos em dois grupos:

Controles sequenciais

Elementos utilizados para representar fluxos de controle sequenciais.

Controles paralelos

Elementos utilizados para representar fluxos de controle paralelos.

Veremos mais detalhes de cada tipo a seguir.

Fluxo de controle sequencial

Os elementos de um diagrama de atividade usados no controle sequencial são:

Estado ação e estado atividade

Um diagrama de atividades exibe os passos de uma computação. Cada estado corresponde a um dos passos da computação, em que o sistema está realizando algo. Um estado em um diagrama de atividade pode ser um estado atividade ou um estado ação. O primeiro leva um certo tempo para ser finalizado. Já o segundo é realizado instantaneamente.

Estados inicial e final e condição de guarda

Assim como no diagrama de estados, um diagrama de atividade deve ter um estado inicial; ele pode ter também vários estados finais e guardas associados a transições. Um diagrama de atividade pode não ter estado final, o que significa que o processo ou procedimento sendo modelado é cíclico.

Transição de término

Uma transição de término liga um estado a outro. Essa transição significa o término de um passo e o consequente início do outro. Observe que, em vez de ser disparada pela ocorrência de um evento (como nos diagramas de estados), essa transição é disparada pelo término de um estado de ação. Pode haver uma transição rotulada com a condição de guarda especial [else], o que significa que, se todas as demais condições de guarda foram avaliadas como falsas, a transição associada a essa guarda especial é disparada.

Pontos de ramificação

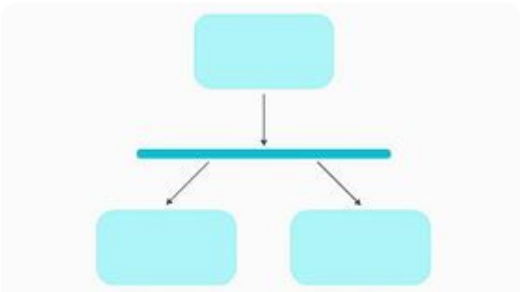
Um ponto de ramificação possui uma única transição de entrada e várias de saída. Para cada transição de saída, há uma condição de guarda associada. Quando o fluxo de controle chega a um ponto de ramificação, uma e somente uma das condições de guarda deve ser verdadeira.

Ponto de união

Um ponto de união reúne diversas transições que, direta ou indiretamente, têm um ponto de ramificação em comum.

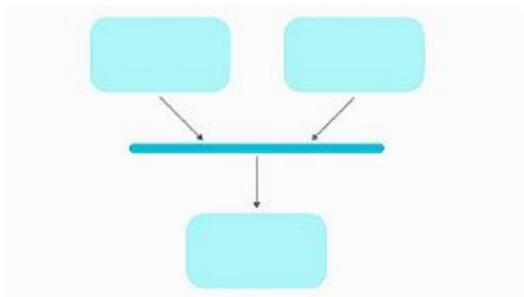
Fluxo de controle paralelo

Um diagrama de atividade pode conter fluxos de controle paralelos. Isso significa que é possível haver dois ou mais fluxos de controle sendo executados simultaneamente em um diagrama de atividades. Para sincronizar dois ou mais fluxos paralelos, as barras de sincronização são utilizadas. Há dois tipos de barra de sincronização:



Barra de bifurcação (fork)

Uma barra de bifurcação recebe uma transição de entrada e cria dois ou mais fluxos de controle paralelos. A partir desse momento, cada um dos fluxos criados é executado independentemente e em paralelo com os demais.



Barra de junção (join)

Uma barra de junção recebe duas ou mais transições de entrada e une os fluxos de controle em um único fluxo. Seu objetivo é sincronizar fluxos de controle paralelos já existentes. As transições de saída da barra de junção só são disparadas após o disparo de todas as transições de entrada.

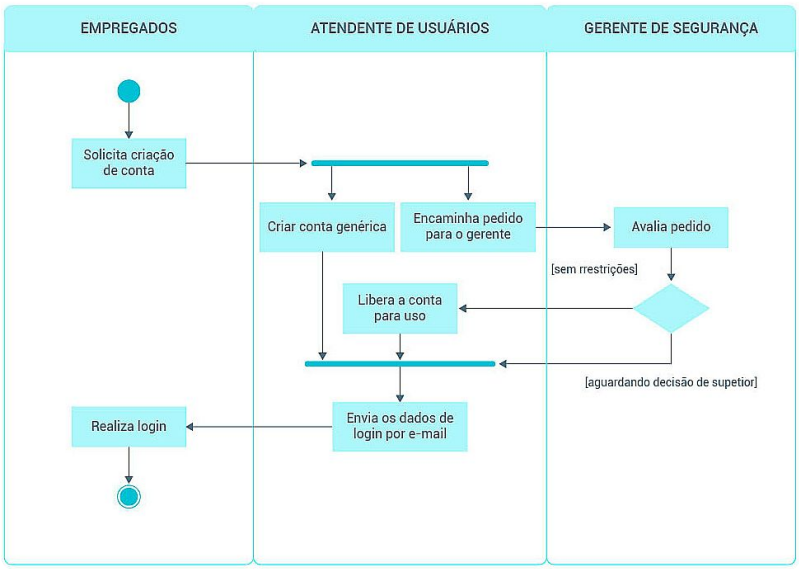
Raias

Algumas vezes, as atividades de um processo podem ser distribuídas por vários agentes que o executarão. Isso normalmente ocorre em processos de negócio de uma organização, em que as tarefas são executadas por pessoas ou departamentos diversos. Nesses casos, o processo pode ser representado em um diagrama de atividades com o uso de raias de natação. As raias de natação dividem o diagrama de atividades em

compartimentos. Cada compartimento contém atividades que são realizadas por um agente específico. Além disso, as entidades de cada compartimento podem estar realizando atividades em paralelo.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de atividades com raias e fork.

Modelagem da lógica de uma operação complexa

Quando um sistema de software é adequadamente decomposto em seus objetos constituintes e as responsabilidades de cada objeto estão bem definidas, a maioria das operações é bastante simples. Essas operações não necessitam de modelagem gráfica para serem entendidas. No entanto, em alguns casos, notadamente quando uma operação de uma classe de controle implementa uma regra de negócio, pode haver a necessidade de se descrever a lógica dessa operação ou da própria regra de negócio. Diagramas de atividade também podem ser utilizados com esse objetivo, nisso se assemelhando a um fluxograma.

Atividade 4

Os elementos de um diagrama de atividade podem ser divididos em elementos utilizados para representar fluxos de controle sequenciais e elementos utilizados para representar fluxos de controle paralelos. Qual elemento de um diagrama de atividades é utilizado para criar dois ou mais fluxos de controle paralelos?

A Ponto de união

B Barra de bifurcação (fork)

C Ponto de ramificação

D Transição de término

E Barra de junção (join)

A alternativa B está correta.

A barra de bifurcação (fork) é o elemento utilizado para criar dois ou mais fluxos de controle paralelos em um diagrama de atividades. Ao receber uma transição de entrada, ela cria múltiplos fluxos de controle independentes e paralelos. Isso permite que diferentes partes do processo ocorram simultaneamente.

Modelagem de atividades na prática

Assim como o diagrama de estados deve ser criado para classes com comportamento complexo, o diagrama de atividades deve ser criado para os casos de uso com fluxos de controle complexos. Assim como o diagrama de estados é aplicado a apenas uma classe, cada diagrama de atividades deve ser feito para um único caso de uso.

No entanto, o diagrama de atividades pode também ser utilizado para modelar o funcionamento geral de um negócio, oferecendo uma visão de alto nível dos macroprocessos, ou para detalhar a lógica de uma regra de negócio específica, proporcionando uma visão mais aprofundada.

Para nosso exemplo prático de modelagem de atividades, será feita a modelagem de atividades do caso de uso enviar veículo para conserto. Para a confecção do diagrama de atividades, deve ser observada a descrição do caso de uso a ser modelado. Como o caso de uso de exemplo já foi introduzido, vamos apresentar uma descrição para ele, e para o caso de uso encerrar aluguel, do qual ele pode ser acionado por meio de um relacionamento <<include>>.

Assista ao vídeo para compreender como a apresentação deverá permitir a prática da modelagem de atividades, a partir de determinado minimundo, utilizando a UML.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

A seguir, temos as duas descrições dos casos de uso encerrar aluguel e enviar veículo para conserto.

Caso de uso encerrar aluguel

1. Funcionário inicia caso de uso.
2. Sistema solicita código do funcionário.
3. Funcionário fornece seu código.
4. Sistema valida código e solicita CNH do cliente.
5. Funcionário fornece CNH.
6. Sistema disponibiliza aluguel.
7. Funcionário informa encerramento do aluguel, informando possíveis observações. Caso encerramento se deva a um defeito no veículo, o caso de uso enviar veículo para conserto é realizado.

8. Sistema armazena informações e encerra o aluguel.
9. Caso de uso é encerrado.

Caso de uso enviar veículo para conserto

1. Funcionário inicia caso de uso.
2. Se o veículo estiver alugado, o sistema obtém placa do veículo diretamente do registro do aluguel quando o caso de uso é iniciado a partir do caso de uso encerrar aluguel e não executa o próximo passo. Caso contrário, o sistema solicita a placa do veículo.
3. Funcionário fornece placa do veículo.
4. Sistema armazena indisponibilidade do veículo e mostra oficinas disponíveis.
5. Funcionário seleciona oficina.
6. O sistema armazena as informações do conserto e encerra o caso de uso, retornando ao caso de uso encerrar aluguel, caso tenha sido iniciado a partir dele.

Observe que no passo 7 do caso de uso encerrar aluguel pode ocorrer a chamada do encerrar aluguel (relacionamento <>), e o passo 2 desse caso de uso trata dessa chamada. Observando a lógica do encerrar aluguel, é possível identificar:

- Ao receber o veículo com defeito, ele pode estar alugado ou não.
- Se estiver alugado, o aluguel é encerrado.
- O veículo é colocado indisponível.
- A oficina é escolhida.
- O veículo é enviado para a oficina.

O diagrama de atividades correspondente é apresentado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

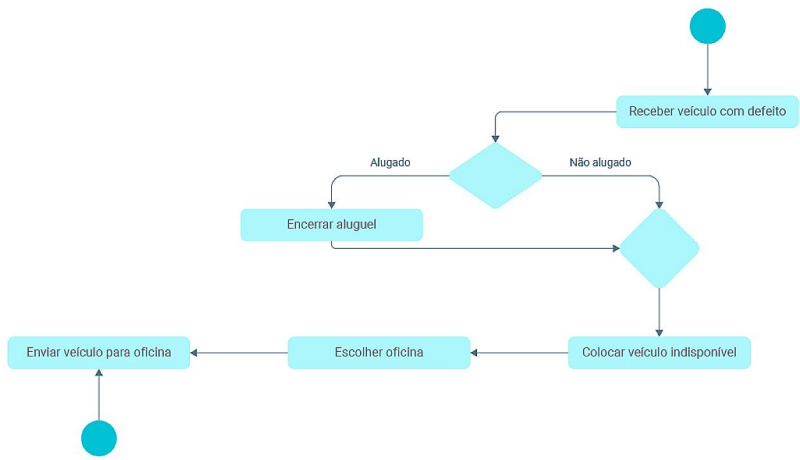


Diagrama de atividades.

Atividade 5

Como engenheiro de software realizando a modelagem de atividades, qual interpretação correta você aplicaria na sua modelagem em relação aos casos de uso encerrar aluguel e enviar veículo para conserto?

A O sistema solicita a CNH do cliente após o funcionário informar o encerramento do aluguel no caso de uso encerrar aluguel.

B No caso de uso enviar veículo para conserto, se o veículo estiver alugado, o sistema obtém a placa diretamente do cadastro do aluguel e não solicita a placa do veículo ao funcionário.

C O relacionamento entre os casos de uso encerrar aluguel e enviar veículo para conserto é do tipo <>.

D No caso de uso encerrar aluguel, o sistema armazena as informações do conserto e encerra o aluguel no passo 7.

E No caso de uso enviar veículo para conserto, o sistema armazena a indisponibilidade do veículo somente após a seleção da oficina.

A alternativa B está correta.

No caso de uso enviar veículo para conserto, se o veículo estiver alugado, o sistema obtém a placa do veículo diretamente do cadastro do aluguel e não executa o passo de solicitar a placa ao funcionário.

Diagrama de componentes

Apresenta o relacionamento entre os diferentes componentes de um sistema de software. Na UML, o termo componente refere-se a um módulo que representa sistemas ou subsistemas com capacidade de interagir. Para isso, existe uma abordagem de desenvolvimento em torno de componentes.

O diagrama de componentes permite documentar como os componentes estão estruturados, organizados e como se relacionam, permitindo uma melhor compreensão e facilitando a sua reutilização. Também serve para modelar arquivos de processos de negócio, descrevendo as partes que participam dele e suas relações.

Assista ao vídeo para compreender o emprego do diagrama de componentes, utilizando a UML, ao longo do processo de desenvolvimento de software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O diagrama de componentes apresenta o relacionamento entre os diferentes componentes de um sistema de software. Na UML, o termo "componente" refere-se a um módulo que representa sistemas ou subsistemas com capacidade de interagir. Para isso, existe uma abordagem de desenvolvimento em torno de componentes. Nesse diagrama, são representados os diferentes componentes de software necessários para que o sistema funcione corretamente.

Comentário

Em uma abordagem de programação orientada a objetos, o desenvolvedor usa o diagrama de componentes para agrupar classes com base em um objetivo comum. Assim, é função do diagrama de componentes documentar como os componentes estão estruturados, organizados e como se relacionam, permitindo uma melhor compreensão e facilitando a sua reutilização. Também serve para modelar arquivos de processos de negócio, descrevendo as partes que participam dele e suas relações.

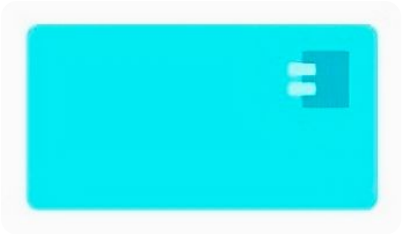
Um diagrama de componentes suporta tanto a especificação dos componentes lógicos, como, por exemplo, componentes de negócio, de processo, entre outros. Os principais benefícios na construção desse diagrama são:

1. Permitir visualizar a estrutura lógica e física do sistema.
2. Identificar os componentes do sistema e como eles se relacionam.
3. Perceber o comportamento do serviço quanto à interface.

Componente

Um componente nada mais é do que uma caixa preta (artefato do sistema) que possui vida autônoma e que, somente por meio de suas interfaces, oferece ou requer serviços de outras caixas pretas, sem que para isso tenha que conhecer seus conteúdos. Um componente é modelado ao longo do ciclo de vida de desenvolvimento, incluindo a concepção da funcionalidade do sistema por meio de casos de uso, definição das classes, entre outros.

Na UML, os componentes também fazem uso de estereótipos. Os mais utilizados são arquivos, tabela, banco de dados, executável e biblioteca. Assim sendo, podemos dizer que um componente é qualquer arquivo que seja necessário à execução do sistema de informação.



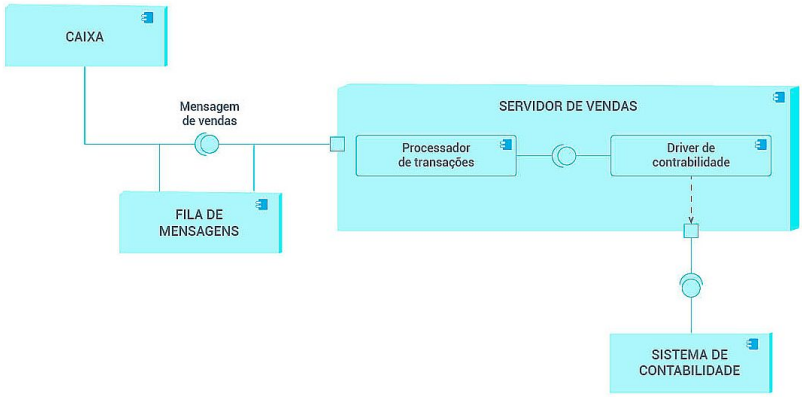
Representação de um componente.

Os componentes são elementos de alto nível que permitem o agrupamento de várias interfaces sem considerá-las um pacote. Cada componente é representado por um retângulo contendo um nome que deve traduzir sua função. Se necessário, pode conter um estereótipo. No canto superior do retângulo, há um ícone que é também um retângulo com dois menores sobrepostos.

Os componentes também podem ser descritos por meio da visão externa usando símbolos de interface colados na caixa do componente. Nesse caso, é interessante anexar uma máquina de estados para interfaces, portas e para o próprio componente para permitir uma descrição mais precisa.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de componentes.

Quando mais detalhes são necessários para a perfeita especificação de um componente, podemos definir uma estrutura interna de partes e conectores, usando o que podemos chamar de visão interna ou “caixa branca”. Nessa estrutura, geralmente estão contidos outros componentes. Esses componentes podem ser implementados por diferentes tipos de arquivo.

Comentário

No diagrama de componentes, esses tipos podem ser destacados por meio de estereótipos, representados por rótulos acima do nome do componente. Nas diretrizes da UML definidas pela OMG, um artefato é dito como sendo definido pelo usuário e representa um elemento concreto do mundo físico. Ele pode ter propriedades ou operações e pode ser instanciado ou associado com outros artefatos.

Interfaces

São coleções de operações que especificam serviços de um componente. É por meio delas que os componentes se comunicam com o mundo externo, seja para oferecer ou receber serviços. Interfaces descrevem o comportamento visível externamente e não especificam qualquer estrutura (não podem incluir atributos), nem qualquer implementação.

> As interfaces fazem as associações entre os componentes do software.

As interfaces podem ser chamadas quando requerem ou esperam serviços. Podem ser usadas opcionalmente por meio de portas em que é possível haver informações adicionais, tais como requisito de desempenho ou políticas que determinem que a interface seja realizada de maneira adequada.

Conectores

São utilizados nos diagramas de componentes para incluir interfaces baseadas em restrições e notações. Existem dois tipos principais de conectores:

Conector assembly: conecta dois componentes para definir que um deles fornece os serviços que o outro requer.

Concetor delegate: indica que um sinal que chega para um componente será transferido para o outro para tratamento ou manipulação. Isso significa que quando colocamos um conector de delegação, estamos dizendo que naquela instância do componente o trabalho não será realizado, mas por outro componente. Esse tipo de conector é bastante usado para modelar decomposição hierárquica em serviços.

Camadas

A arquitetura de sistemas em camadas permite isolar a camada de apresentação das de regras de negócio e de armazenamento de dados. Isso permite aumentar o desempenho do sistema, além de facilitar a manutenção, atualização e substituição de componentes. Isso é muito interessante nos sistemas atuais que se caracterizam muitas vezes por atender grande número de usuários e áreas de negócio.

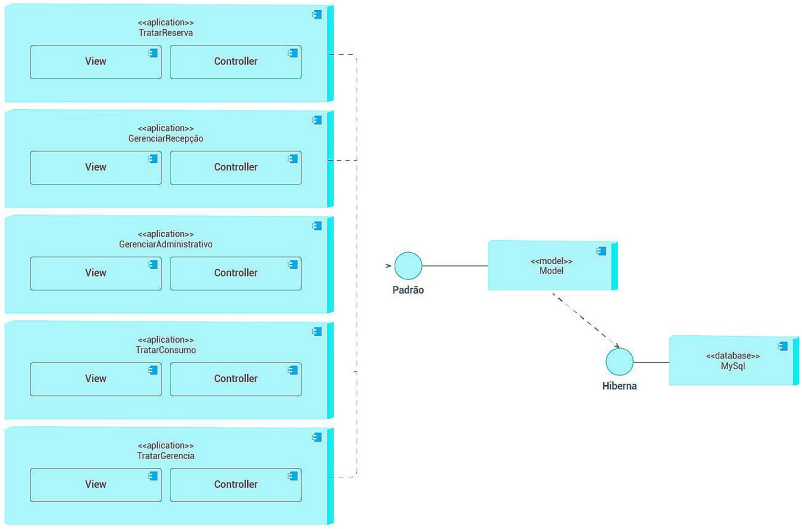
Exemplo

Um exemplo clássico de arquitetura em camadas é o padrão de projeto MVC (Model – View – Controller), em que os componentes da aplicação consistem na interface (View) que invoca o controlador (Controller), o qual, por sua vez, acessa o banco de dados (Model).

Essa separação das camadas de software permite a modularização e o reuso dos componentes do sistema. Um exemplo é ilustrado na imagem a seguir, em uma aplicação hipotética (note que diversas outras formas de componentização poderiam ser adotadas, a critério do projetista).

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de componentes usando MVC.

Atividade 1

Aprendemos que o diagrama de componentes documenta arquivos físicos armazenados em meio digital, representando diversos componentes de software do sistema. Assinale a alternativa que indica de forma mais completa o que é explicitado nesse diagrama.

A Reúso e relações de arquivos.

B Uso e definição dos arquivos nos processos de negócio.

C Estrutura, organização e relacionamentos de arquivos.

D Protótipos de arquivos e suas estruturas.

E Comunicabilidade e organização de arquivos.

A alternativa C está correta.

O diagrama de componentes representa os arquivos do sistema que contêm programas e dados, separando-os de acordo com um padrão de projeto, e mostrando sua organização, estrutura e seus relacionamentos.

Diagrama de implantação

Especifica um conjunto de artefatos de infraestrutura que definem a arquitetura física de execução de sistemas de informação. O diagrama de implantação foca a organização da arquitetura sobre a qual o software será implantado e executado, considerando o hardware, o software básico e as redes de comunicação. Isso inclui as seguintes definições:

- Quais máquinas (computadores, servidores, switches etc.).
- Quais programas de software básico (sistema operacional, sistema gerenciador de banco de dados, browser) serão necessários para suportar o sistema.
- Como essas máquinas serão conectadas, a velocidade de conexão e os protocolos de comunicação utilizados.

O diagrama de implantação descreve a implementação física de informações geradas pelo programa de software em componentes de hardware. Na UML, um diagrama de implantação representa a estrutura física do sistema em si, o que, assim como o diagrama de componentes, pode ser feito de várias formas.

Assista ao vídeo para compreender o emprego do diagrama de implantação, utilizando a UML, ao longo do processo de desenvolvimento de software. Veja também as formas de representação e nós do diagrama de implantação.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Formas de representação

O diagrama de implantação especifica um conjunto de artefatos de infraestrutura que podem ser utilizados para definir a arquitetura física de execução de sistemas de informação. Ele foca a organização da arquitetura sobre a qual o software irá ser implantado e executado. Isso inclui as seguintes definições:

- Hardware.
- Software básico e redes de comunicação, ou seja, quais máquinas (computadores, servidores, switches etc).
- Quais programas de software básico (sistema operacional, sistema gerenciador de banco de dados, browser) serão necessários para suportar o sistema.
- Como essas máquinas serão conectadas.
- Qual será a velocidade de conexão.
- Quais serão os protocolos de comunicação utilizados.

O diagrama de implantação descreve a implementação física de informações geradas pelo programa de software em componentes de hardware. Na UML, um diagrama de implantação representa a estrutura física do sistema em si, o que, assim como o diagrama de componentes, pode ser feito de várias formas.

Caixas tridimensionais

Representam os elementos básicos de software básico ou hardware, ou nós no sistema.



Linhas de nós a nós

Indicam relacionamentos.

Formas menores contidas dentro das caixas tridimensionais
Representam os artefatos de software contidos nos nós.

Nós do diagrama de implantação

São definidos de maneira aninhada e representam dispositivos de hardware e ambientes de execução de software. São os elementos mais básicos do diagrama de implantação; podem representar:

- Computadores pessoais ou servidores de bancos de dados.
- Servidores de aplicação ou de internet.
- Sistemas operacionais.
- Browser.
- Sistemas gerenciadores de bancos de dados etc.

Eles podem ser compostos de outros nós, sendo comum encontrar um nó que representa um item de hardware contendo outros nós que representam ambientes de execução.

O nó representa um elemento físico capaz de oferecer recursos computacionais e, geralmente, possui pelo menos memória e processador. Também serve para representar topologias de rede por meio de ligações entre instâncias de nós. Podem ser conectados para representar uma topologia de redes usando caminhos de comunicação que podem ser definidos como servidor de aplicação, servidor web, entre outros.



Atenção

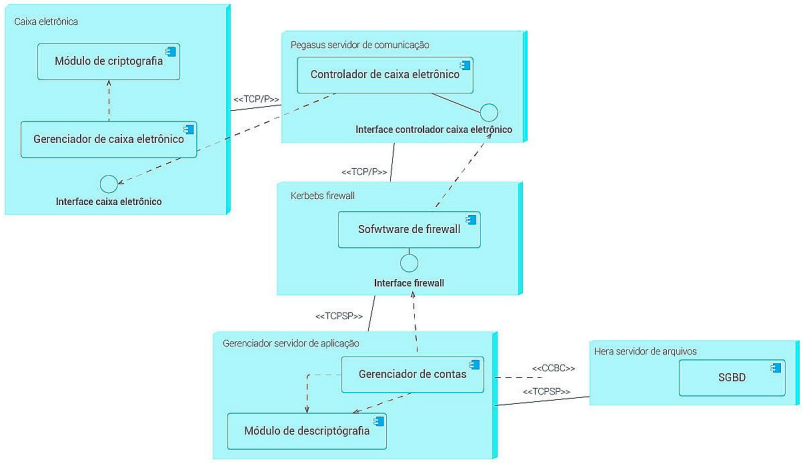
A elaboração do diagrama de implantação depende do porte do sistema a ser desenvolvido. Por exemplo, não faz muito sentido para um sistema que será executado dentro de um único computador. Em outros casos, porém, ele pode ser muito útil, principalmente para ser utilizado como meio de comunicação entre a equipe de desenvolvimento e de infraestrutura, pois servirá de base para a equipe de infraestrutura instalar e configurar servidores, gerenciadores de bancos de dados, entre outros.

A imagem a seguir ilustra um diagrama de implantação.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de diagrama de implantação.

Podem ser usados estereótipos que são, como em outros diagramas da UML, uma maneira gráfica ou textual de diferenciar componentes dos diagramas (classes, casos de uso, associações etc.). Podem ser representados por meio de textos escritos entre dois sinais de maior e menor quando não modificam o modelo original do componente (<<>>) e, também, por meio de um novo componente visual, porém padronizado pela UML, como os nós que podem conter os seguintes estereótipos: **<>, <>, <>**.

Device

Representa um dispositivo computacional com capacidade de processamento em que artefatos podem ser implantados para execução. Os dispositivos podem ser complexos, isto é, podem ser constituídos por outros dispositivos. Para representar um device, utilizamos o estereótipo <>.

Artefatos

Representam objetos concretos do mundo físico, usados e/ou resultantes de um processo de desenvolvimento. São exemplos de artefatos: programas fontes e executáveis, uma tabela de bancos de dados, scripts, um documento do sistema, uma mensagem de correio eletrônico etc. Para representar um artefato, é utilizado o estereótipo <>. Artefatos são sempre implantados em nós.

Ambiente de execução

Representam uma subclasse de um nó. São exemplos de ambiente de execução: sistemas operacionais, sistemas gerenciadores de bancos de dados etc. Para representar um ambiente de execução utilizamos o estereótipo <>.

Associação

Os nós são conectados por meio de caminhos de comunicação para criar sistemas em rede, ou seja, possuem ligações entre si de forma que possam se comunicar e trocar informações. Essas ligações são chamadas de associações e são representadas por segmentos de reta ligando um nó a outro. Uma associação pode conter estereótipos utilizados para determinar, por exemplo, o tipo de protocolo de comunicação usado entre os nós.

Atividade 2

No diagrama de implantação, temos o conceito de nó e como este é utilizado. É correto afirmar que os nós representam:

A Qualquer dispositivo de hardware ou ambiente de execução de software.

B Classes de componentes ou dispositivos de hardware.

C Memória e processador.

D Servidores onde acontece a execução dos componentes do software.

E Os componentes de um software.

A alternativa A está correta.

Os nós não representam um único tipo de artefato de infraestrutura do sistema. São tipicamente definidos de maneira aninhada e representam tanto dispositivos de hardware, quanto ambientes de execução de software básico, como sistemas operacionais, sistemas gerenciadores de banco de dados, navegadores de web etc.

Considerações finais

O que você aprendeu neste conteúdo?

- Modelagem de interação com os diagramas de sequência e de comunicação.
- Modelagem de classes utilizando o diagrama de classes de projeto.
- Modelagem de estados aplicando o diagrama de transição de estados.
- Modelagem de atividades empregando o diagrama de atividades.
- Emprego dos diagramas de componentes e de implantação.

Explore +

Para aprofundar mais seu conhecimento sobre o conteúdo apresentado.

- Veja:O canal do YouTube ***Estudo na Web***, que tem como principal objetivo desmistificar e tornar mais fácil o aprendizado com exemplos simples e exercícios práticos, diversos materiais sobre UML, inclusive alguns específicos sobre os diagramas apresentados aqui.

Leia:

- O artigo ***Os principais diagramas da UML*** – Resumo rápido, apresentado pela revista Profissionais TI, da organização de mesmo nome, que traz um conjunto de diversas ferramentas que podem ser utilizadas para modelagem dos diagramas apresentados neste conteúdo.

Referências

BEZERRA, E. **Princípios de análise e projeto de sistemas com UML**. Rio de Janeiro, Brasil: Campus, 2015.

OMG. **Unified Modeling Language** - Versão 2.5. OBJECT MANAGEMENT GROUP, 2015. Consultado na internet em: 23 jan. 2025.